למידת מכונה

רשתות עצביות

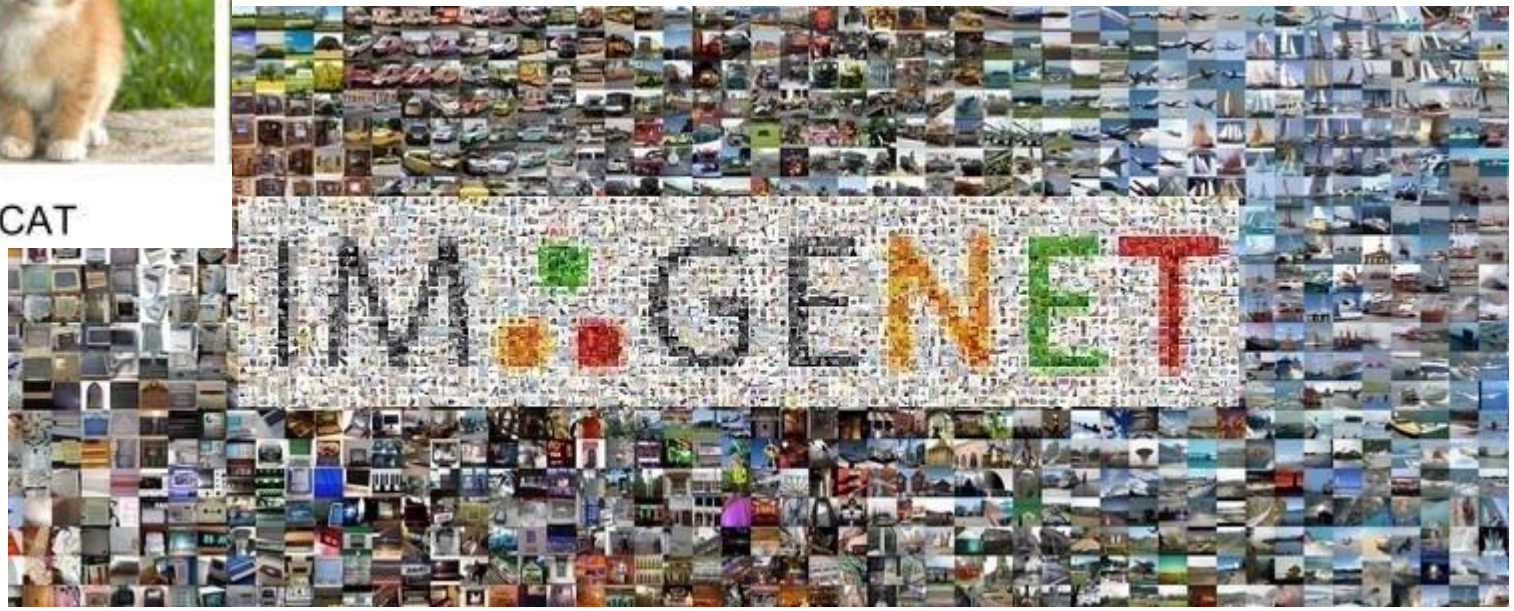
Neural Networks

Image Classification

Classification



CAT



https://medium.com/syncedreview/sensetime-trains-imagenet-alexnet-in-record-1-5-minutes-e944ab049b2c

IMAGENET

IMAGENET

14,197,122 images, 21841 synsets indexed

Explore Download Challenges Publications Updates About

Not logged in. Login | Signup

ImageNet is an image database organized according to the WordNet hierarchy (currently only the nouns), in which each node of the hierarchy is depicted by hundreds and thousands of images. Currently we have an average of over five hundred images per node. We hope ImageNet will become a useful resource for researchers, educators, students and all of you who share our passion for pictures.

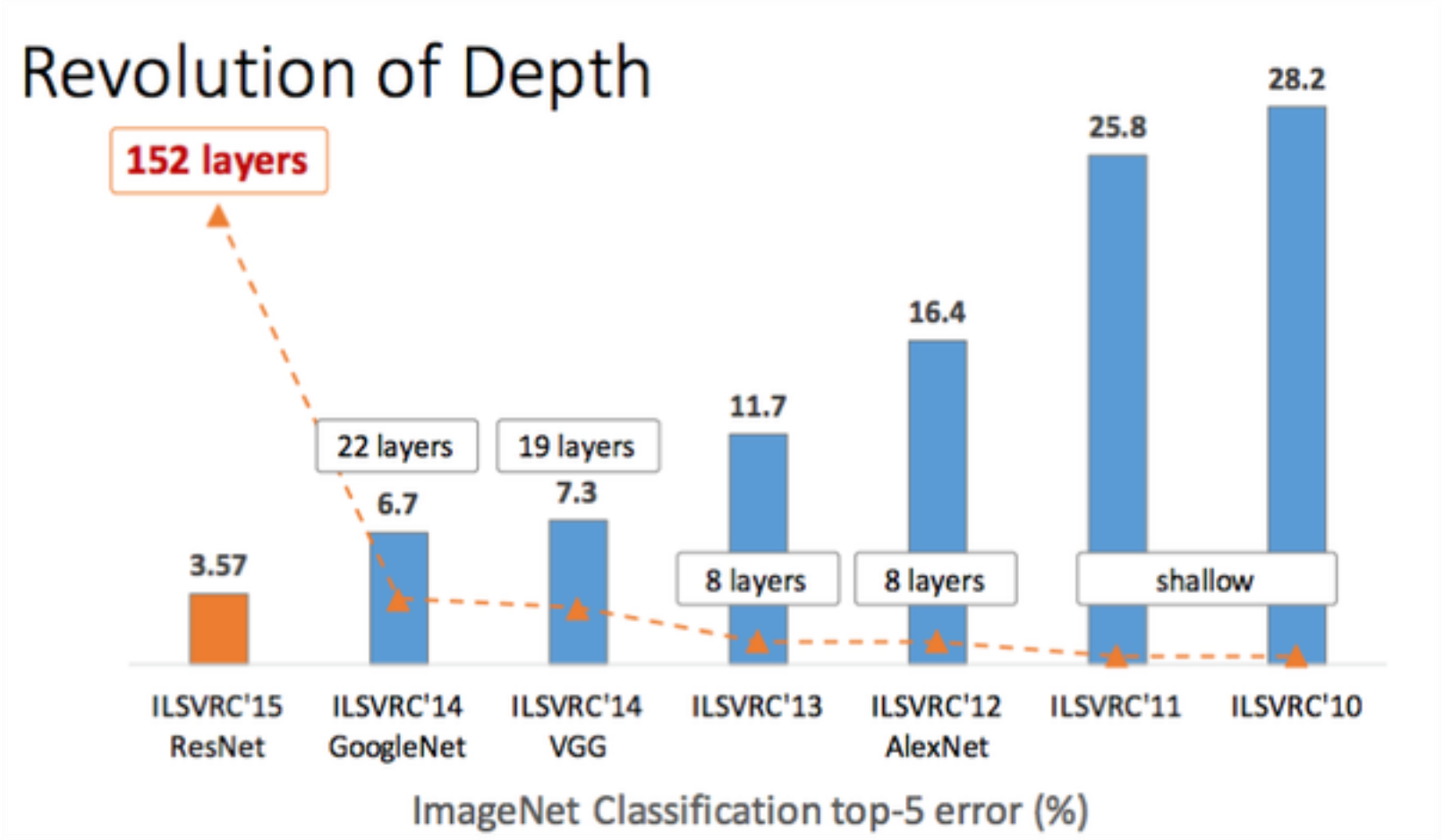
Click here to learn more about ImageNet, Click here to join the ImageNet mailing list.



- Full color, (224x224)

http://www.image-net.org/

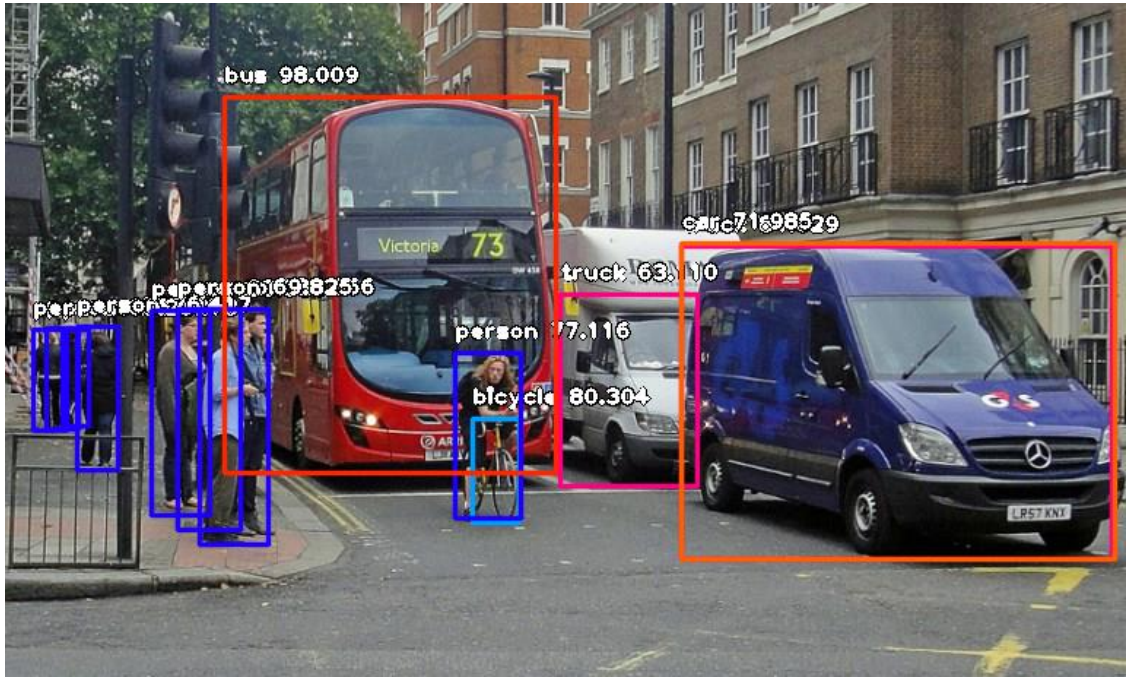
Perspective history



https://towardsdatascience.com/learning-computer-vision-41398ad9941f

Object Detection





https://towardsdatascience.com/object-detection-with-10-lines-of-code-d6cb4d86f606

Image Segmentation





https://medium.com/nanonets/how-to-do-image-segmentation-using-deep-learning-c673cc5862ef

Automatic Colorization



https://richzhang.github.io/colorization/

Automatic Image Caption Generation

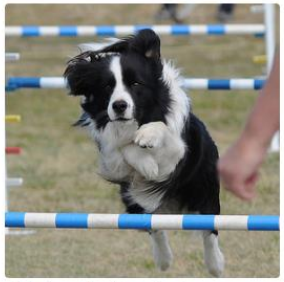



"man in black shirt is playing guitar."


"construction worker in orange safety vest is working on road."


"two young girls are playing with lego toy."


"girl in pink dress is jumping in air."


"black and white dog jumps over bar."


"young girl in pink shirt is swinging on swing."

https://www.analyticsvidhya.com/blog/2018/04/solving-an-image-captioning-task-using-deep-learning/

https://machinelearningmastery.com/inspirational-applications-deep-learning/

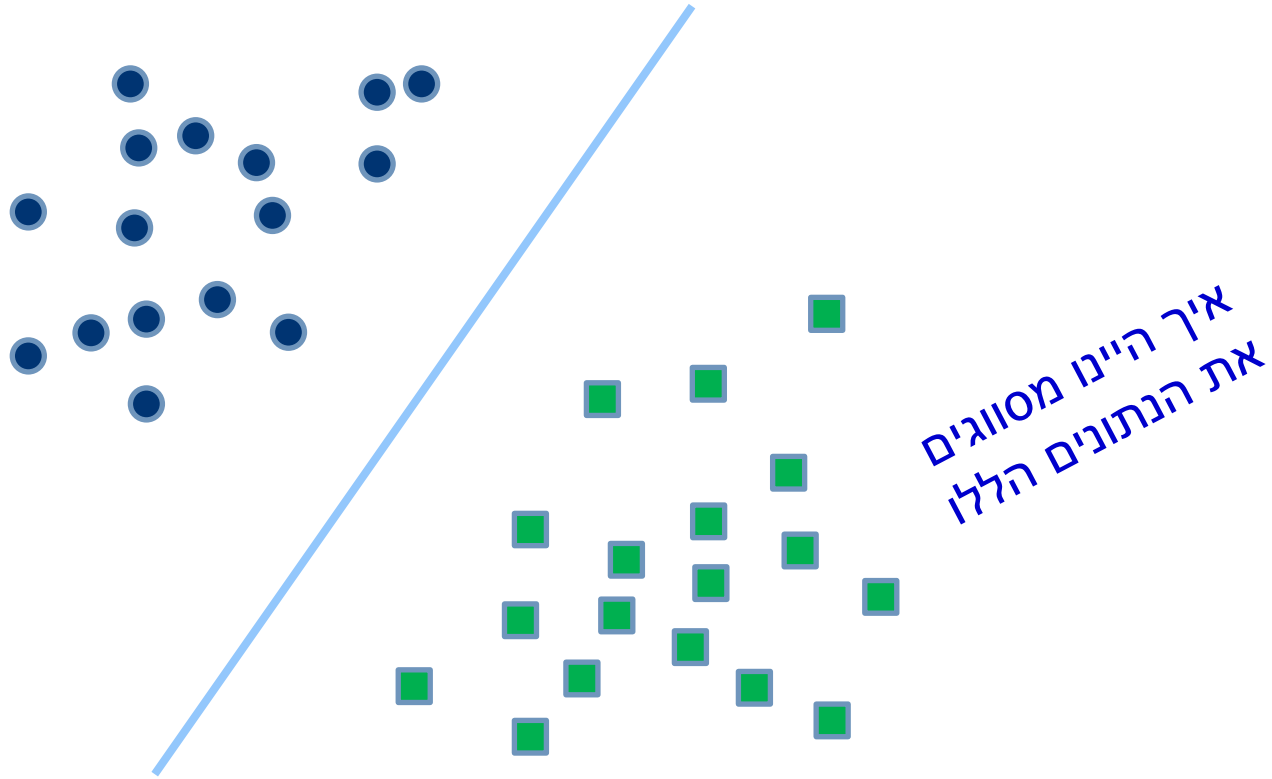
מסווג לינארי



איך היינו מסווגים
את הנתונים הללו

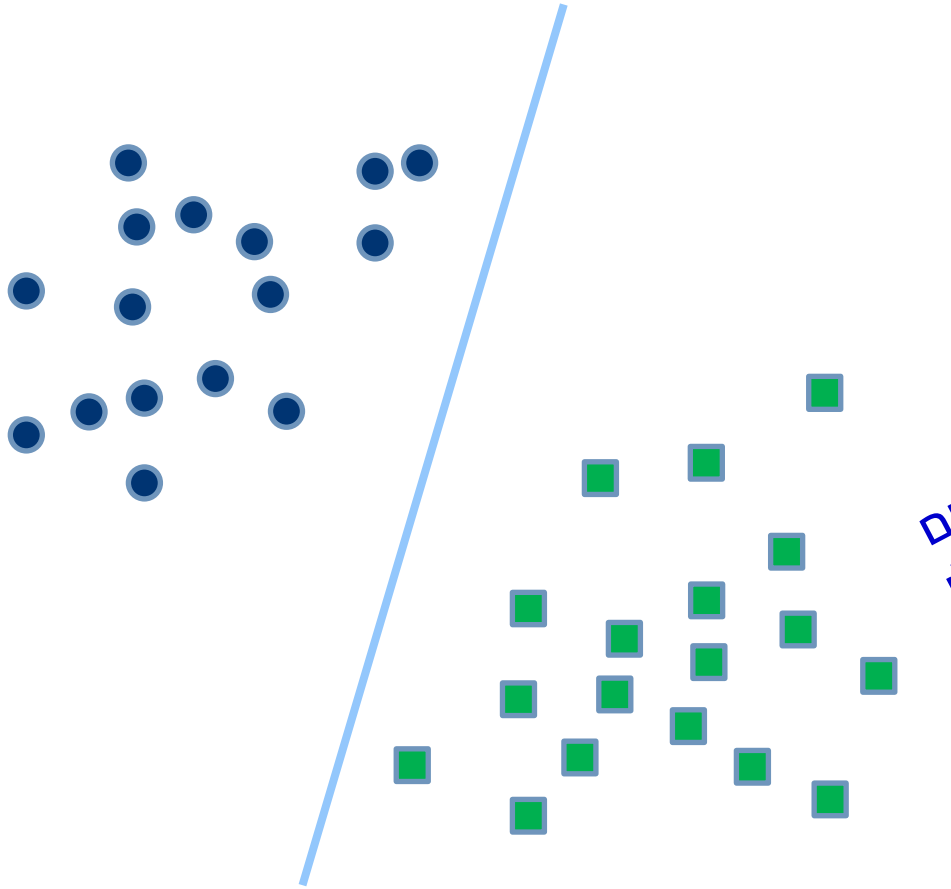
9

מסווג לינארי



10

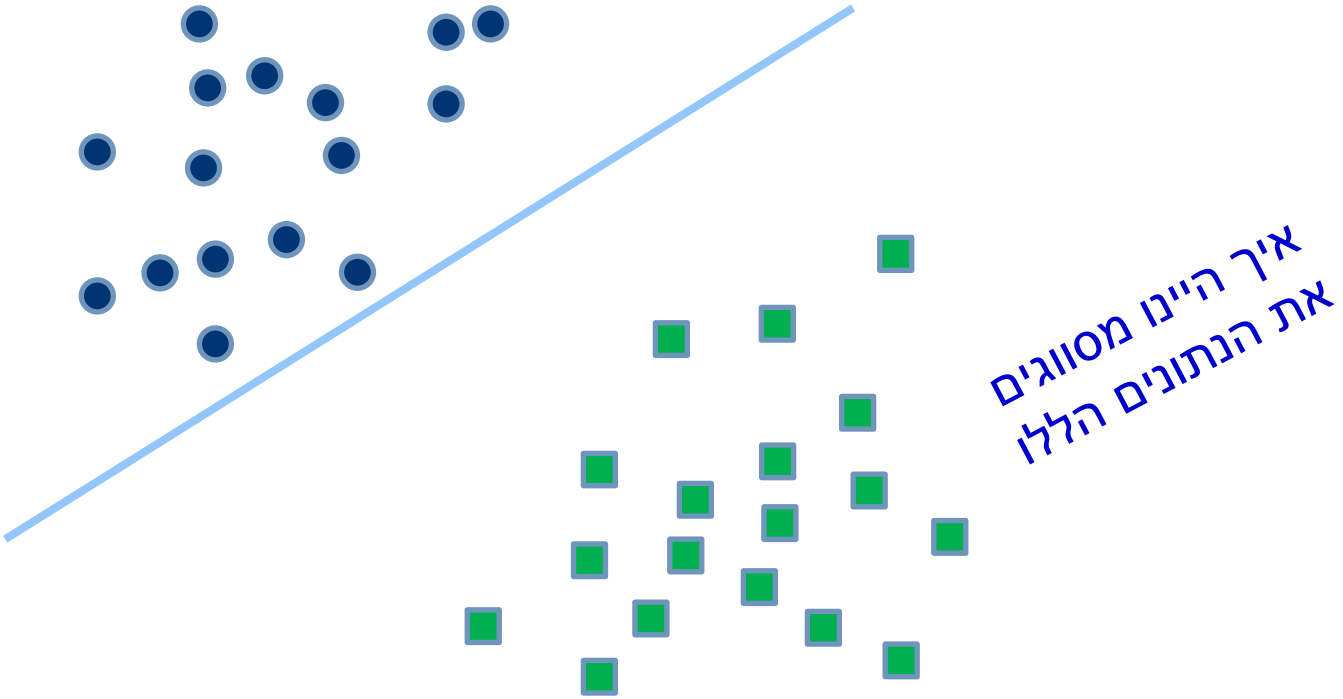
מסווג לינארי



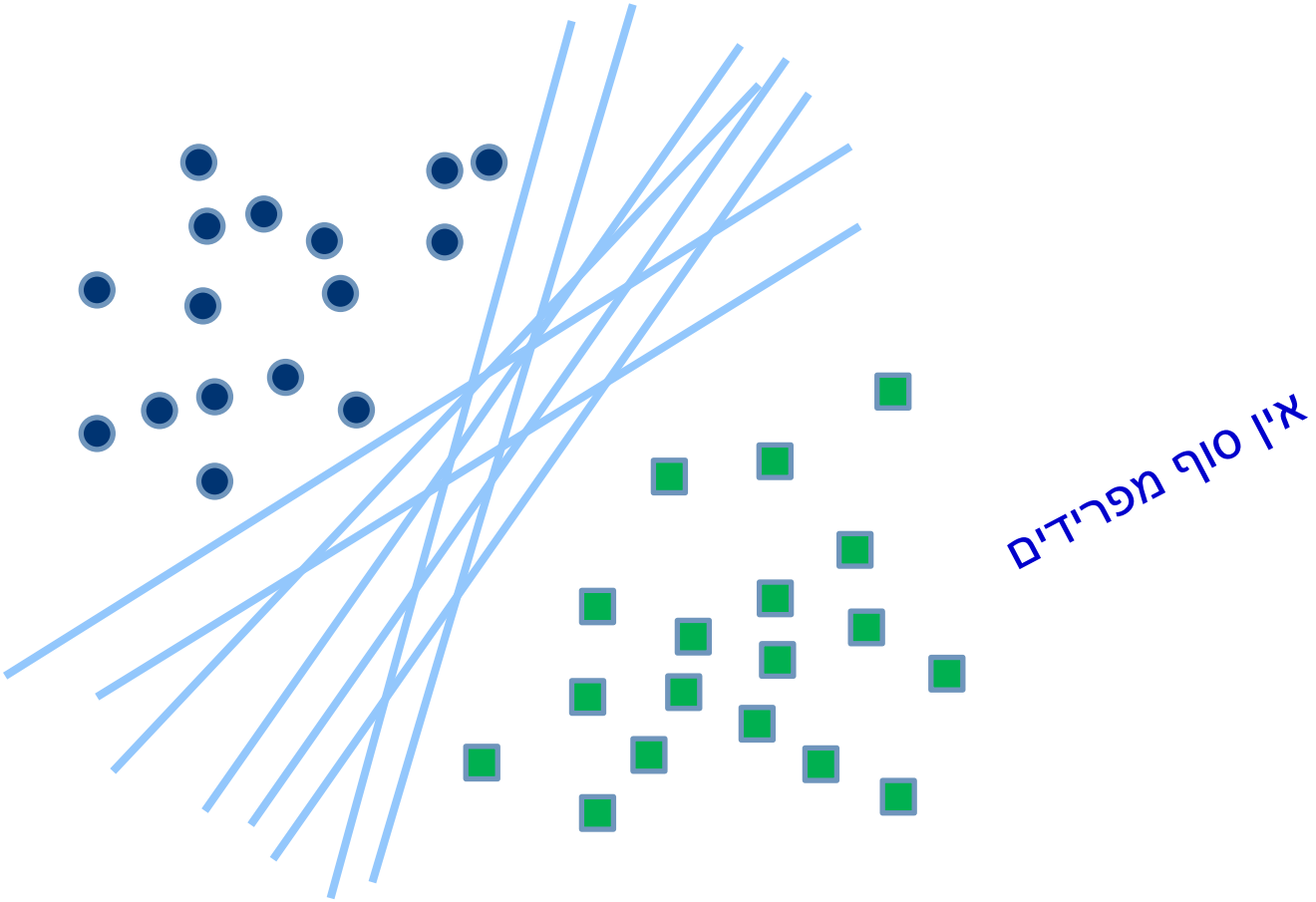
איך היינו מסווגים
את הנתונים הללו

11

מסווג לינארי



מסווג לינארי

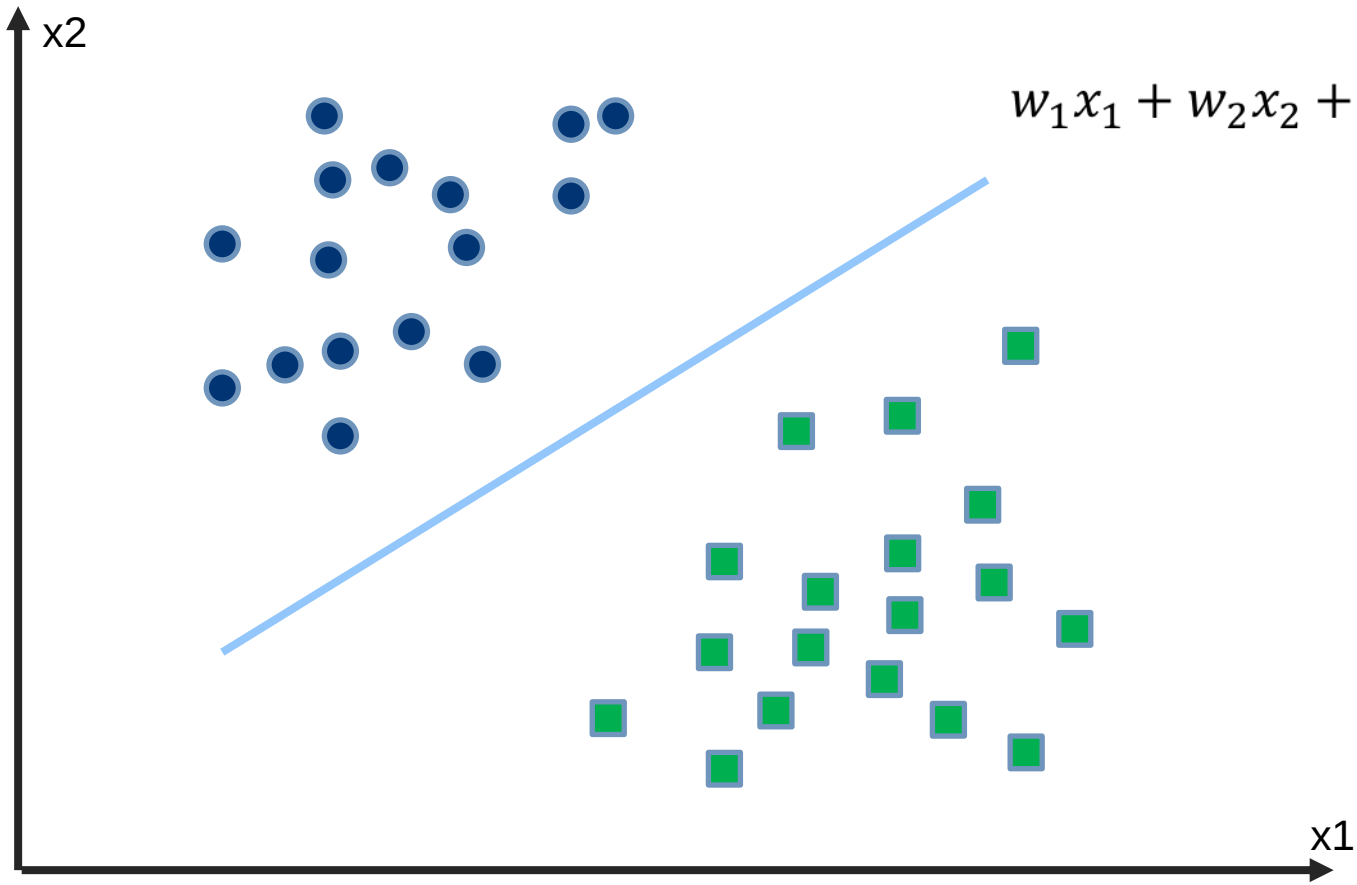


13

מסווג לינארי

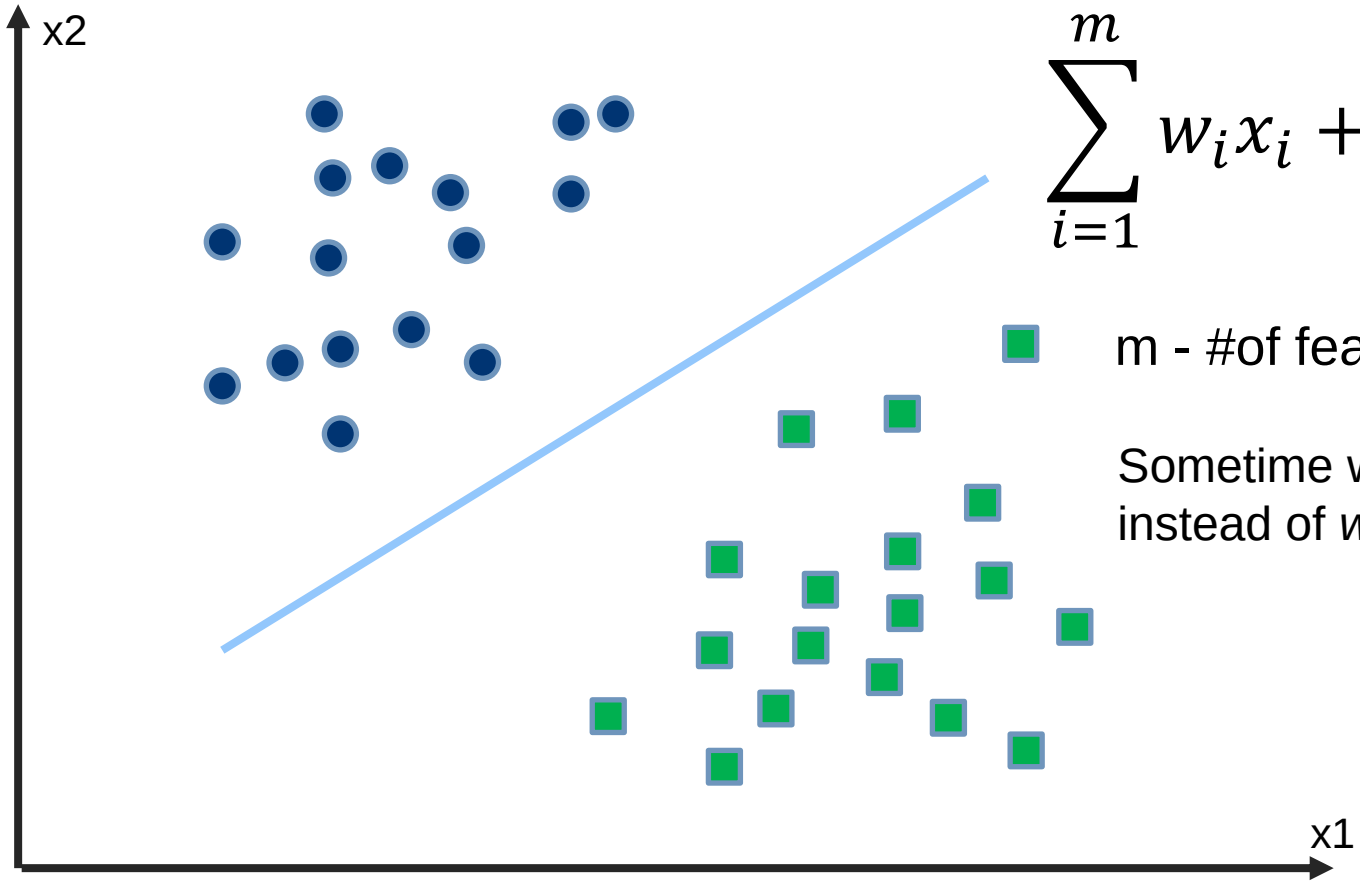
$$ax_1 + bx_2 + c = 0$$

$$w_1x_1 + w_2x_2 + w_0 = 0$$



14

מסווג לינארי – רב מימדי

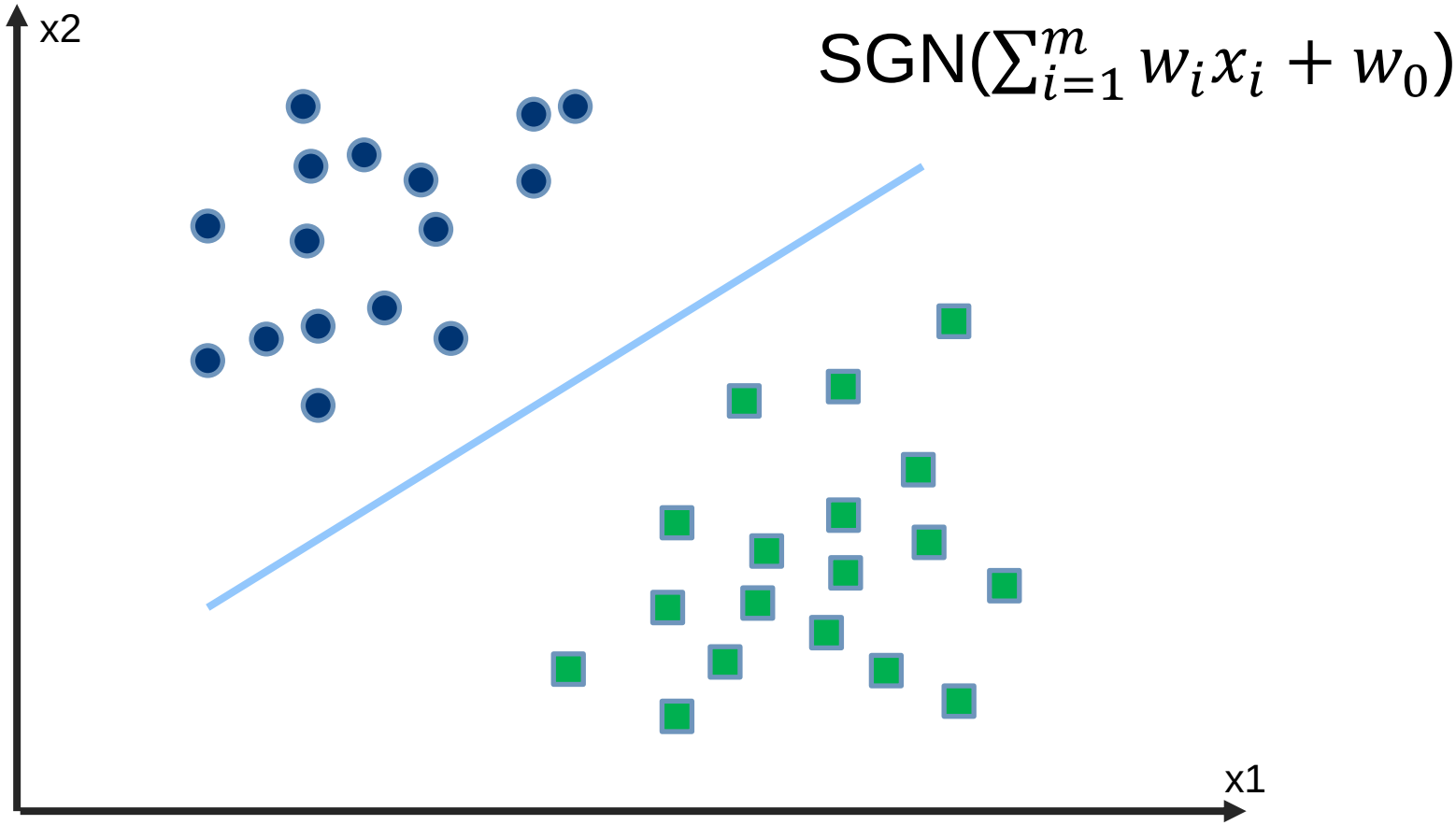


$$\sum_{i=1}^{m} w_i x_i + w_0$$

m - #of features

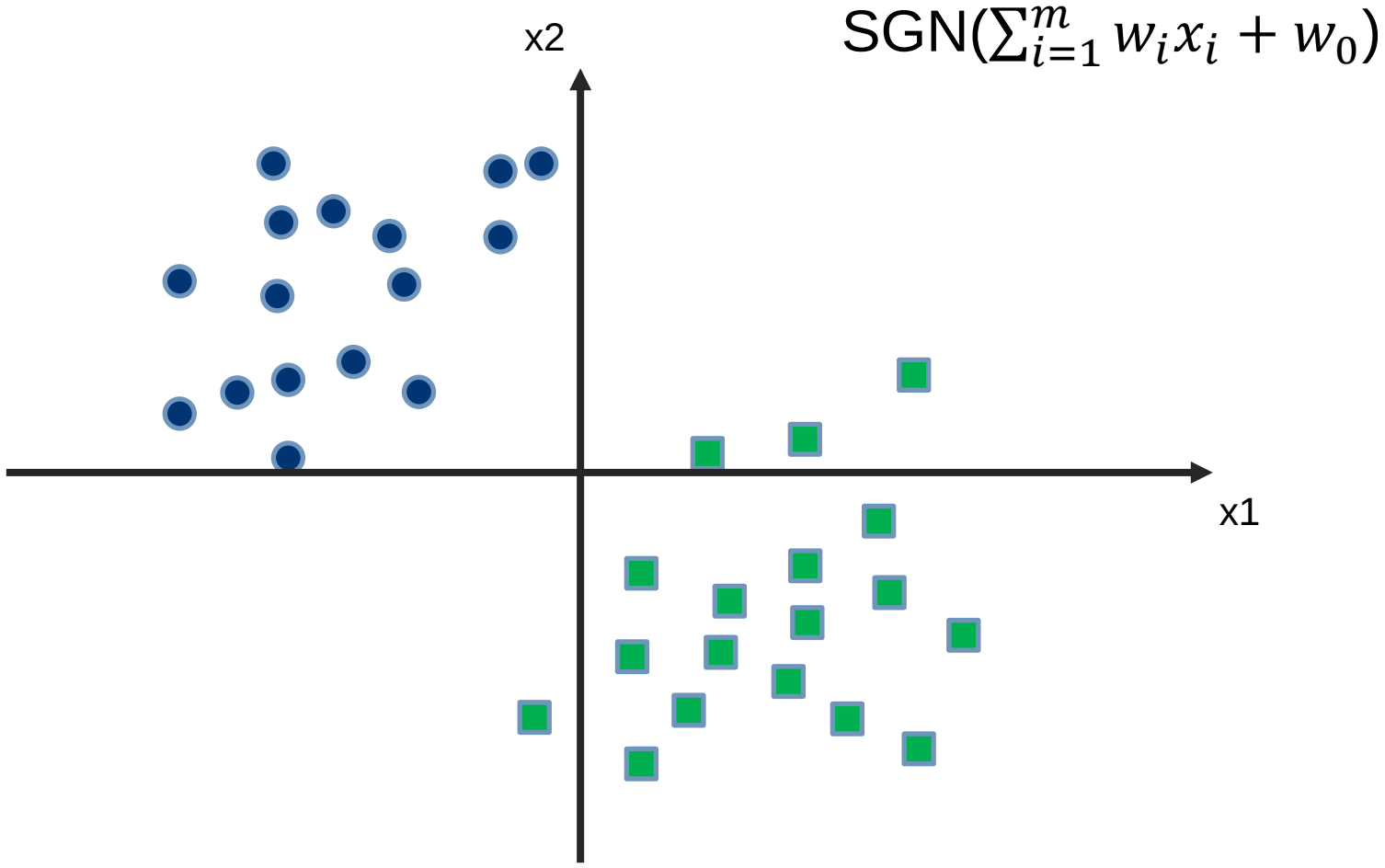
Sometime we use θ instead of w

מסווג לינארי – משוואת הסיווג

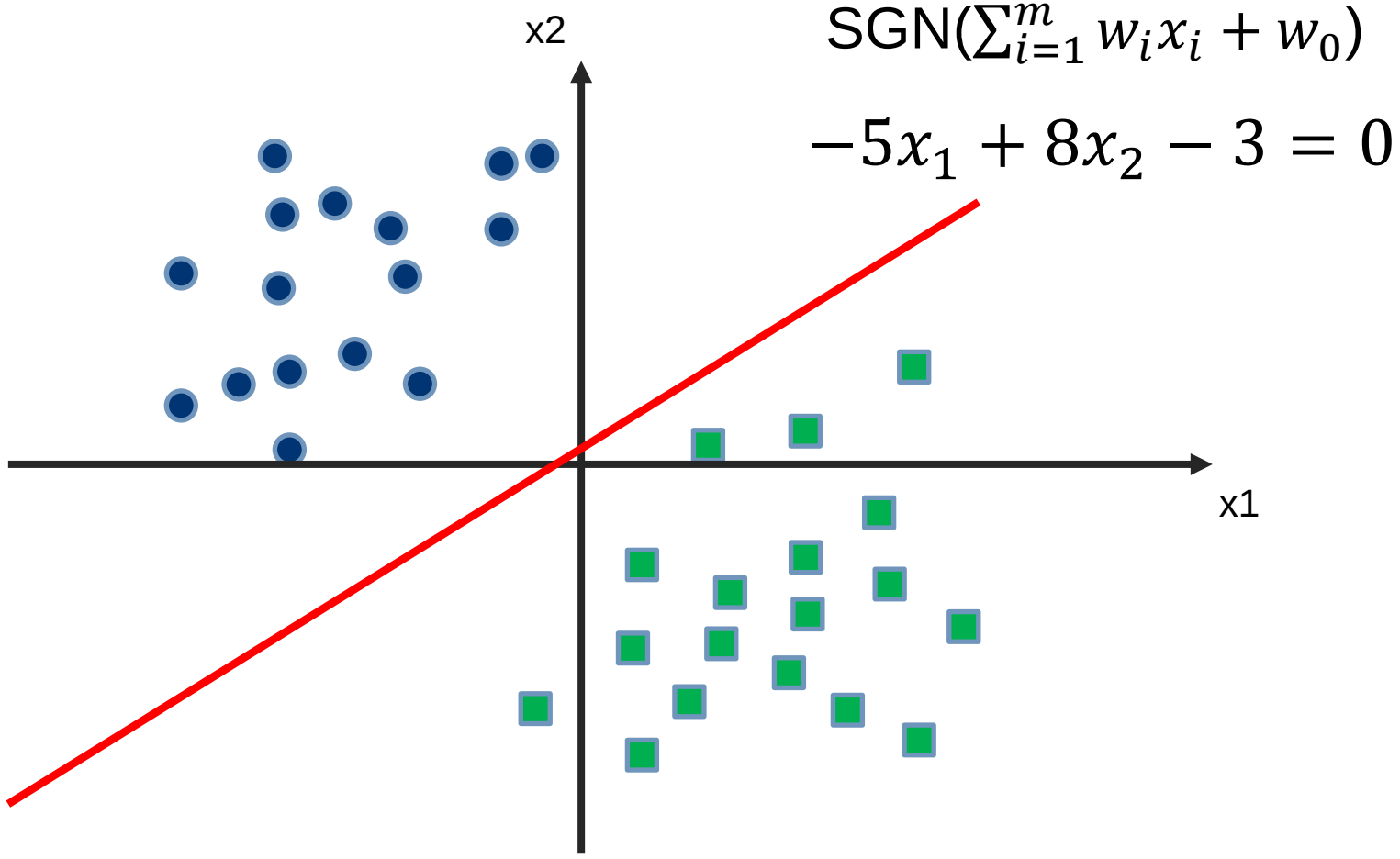


$$\mathrm{SGN}\left(\sum_{i=1}^{m} w_i x_i + w_0\right)$$

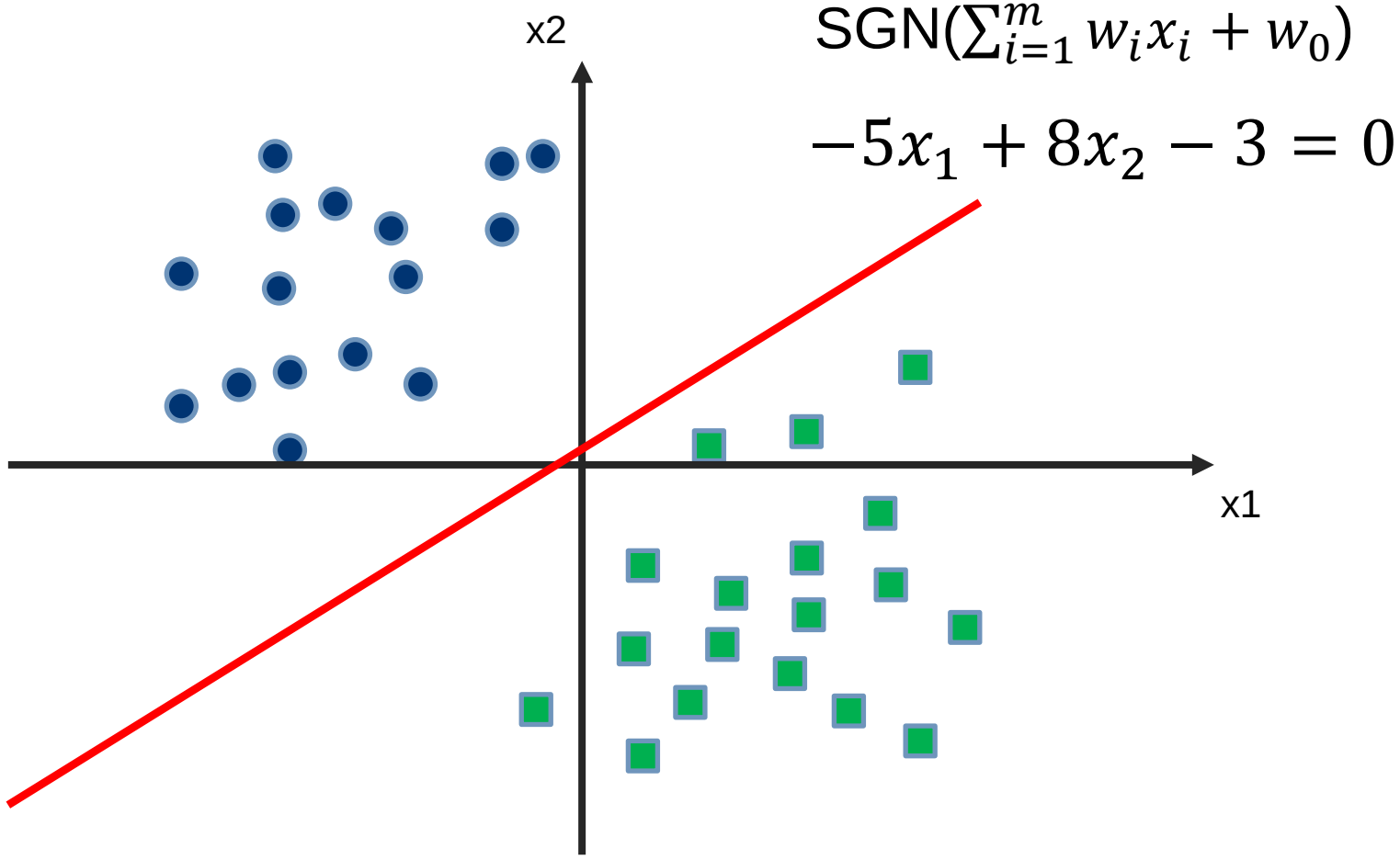
מסווג לינארי – משוואת הסיווג



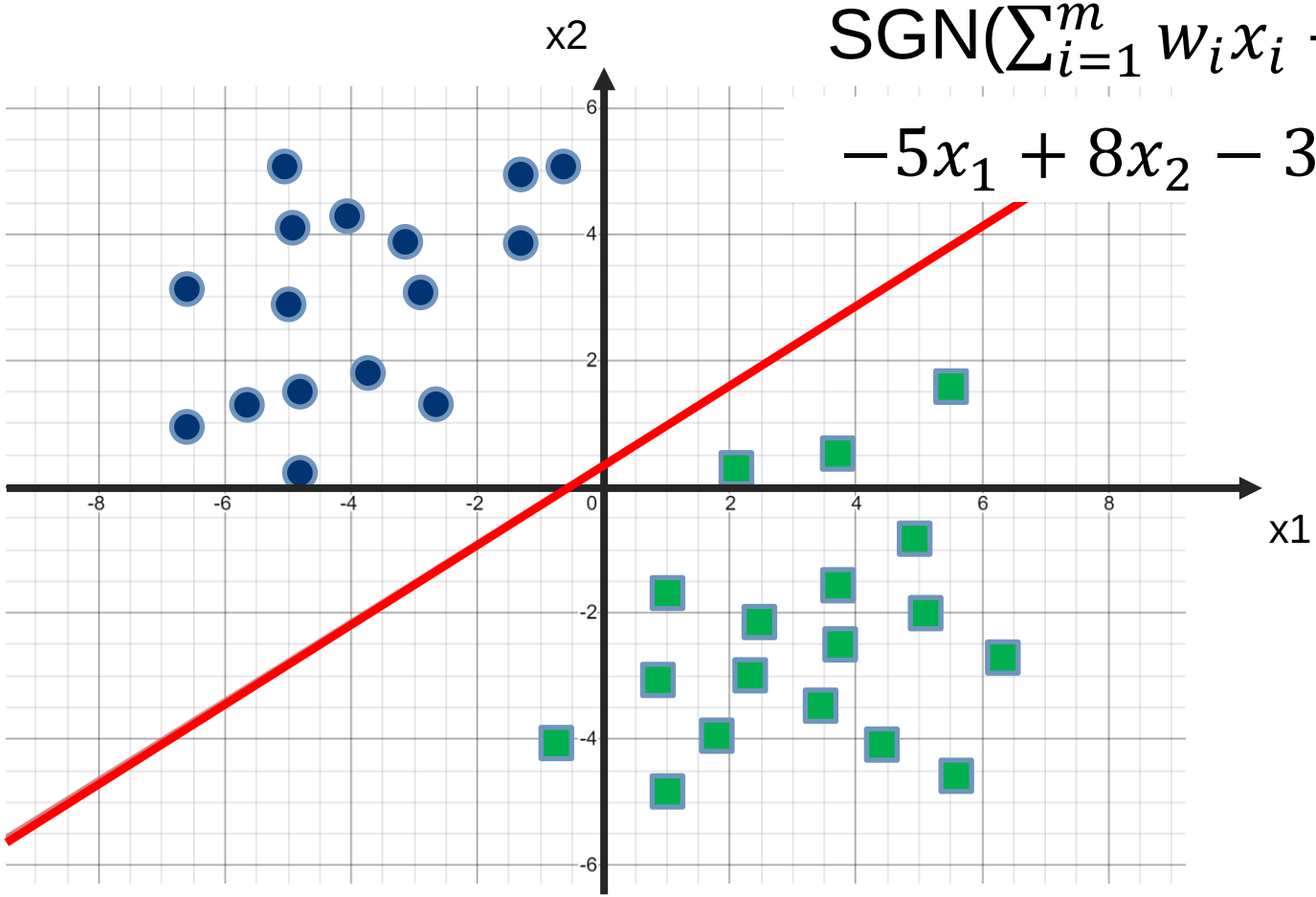
מסווג לינארי – משוואת הסיווג



מסווג לינארי – משוואת הסיווג



מסווג לינארי – משוואת הסיווג



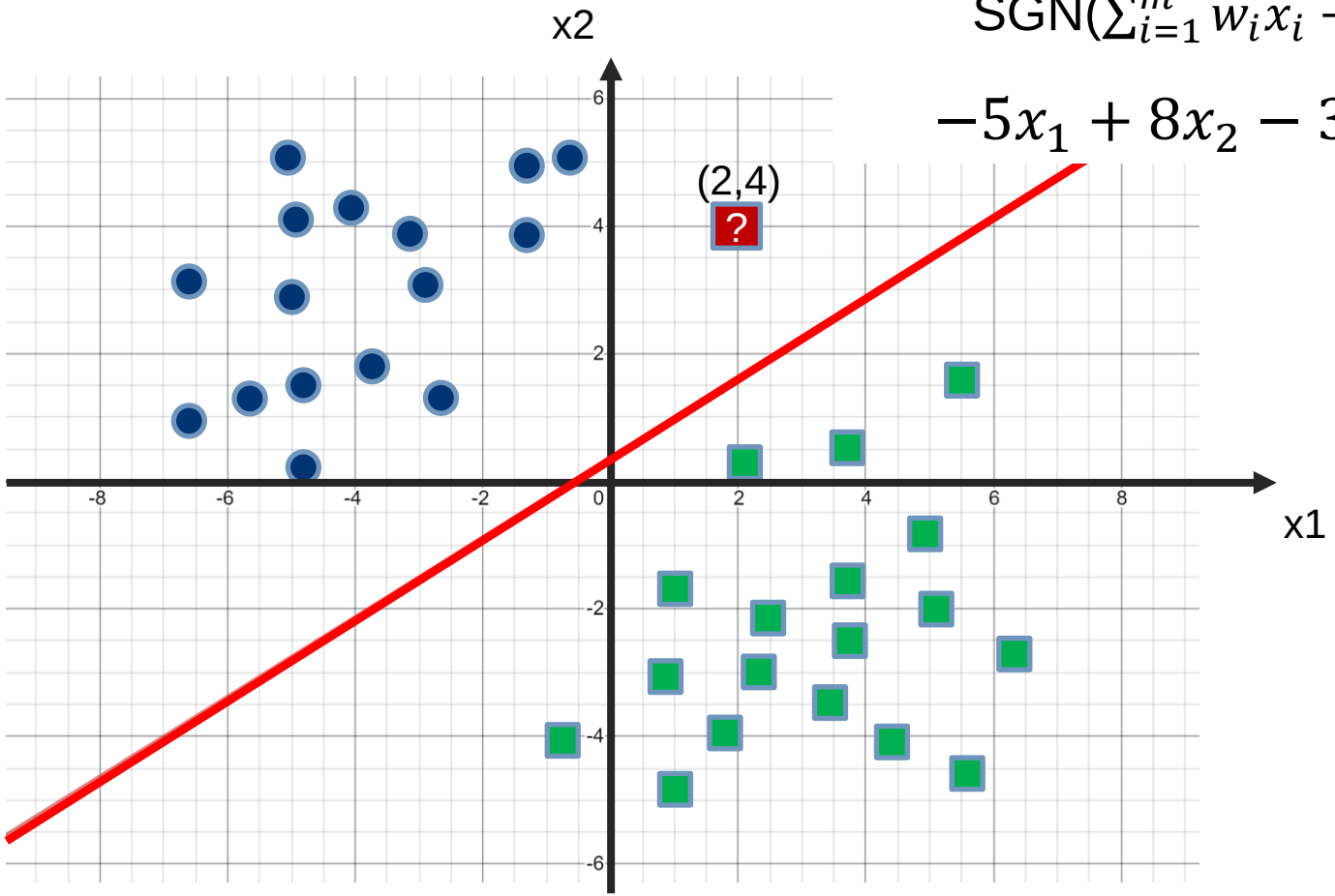
$$\mathrm{SGN}\left(\sum_{i=1}^{m} w_i x_i + w_0\right)$$

$$-5x_1 + 8x_2 - 3 = 0$$

מסווג לינארי – משוואת הסיווג

$$\mathrm{SGN}\left(\sum_{i=1}^{m} w_i x_i + w_0\right)$$

$$-5x_1 + 8x_2 - 3 = 0$$



מסווג לינארי

Hyperplane equation : $-5x_1 + 8x_2 - 3 = 0$

Vector to classify : $(x_1 = 2, x_2 = 4)$

מסווג לינארי

Hyperplane equation : $-5x_1 + 8x_2 - 3 = 0$

Vector to classify : $(x_1 = 2, x_2 = 4)$

$(\quad w_1 = -5 \quad w_2 = +8)$ $\qquad Threshold = 3$

$(\quad x_1 = 2, \qquad x_2 = 4)$

מסווג לינארי

Hyperplane equation : $-5x_1 + 8x_2 - 3 = 0$

Vector to classify : $(x_1 = 2, x_2 = 4)$

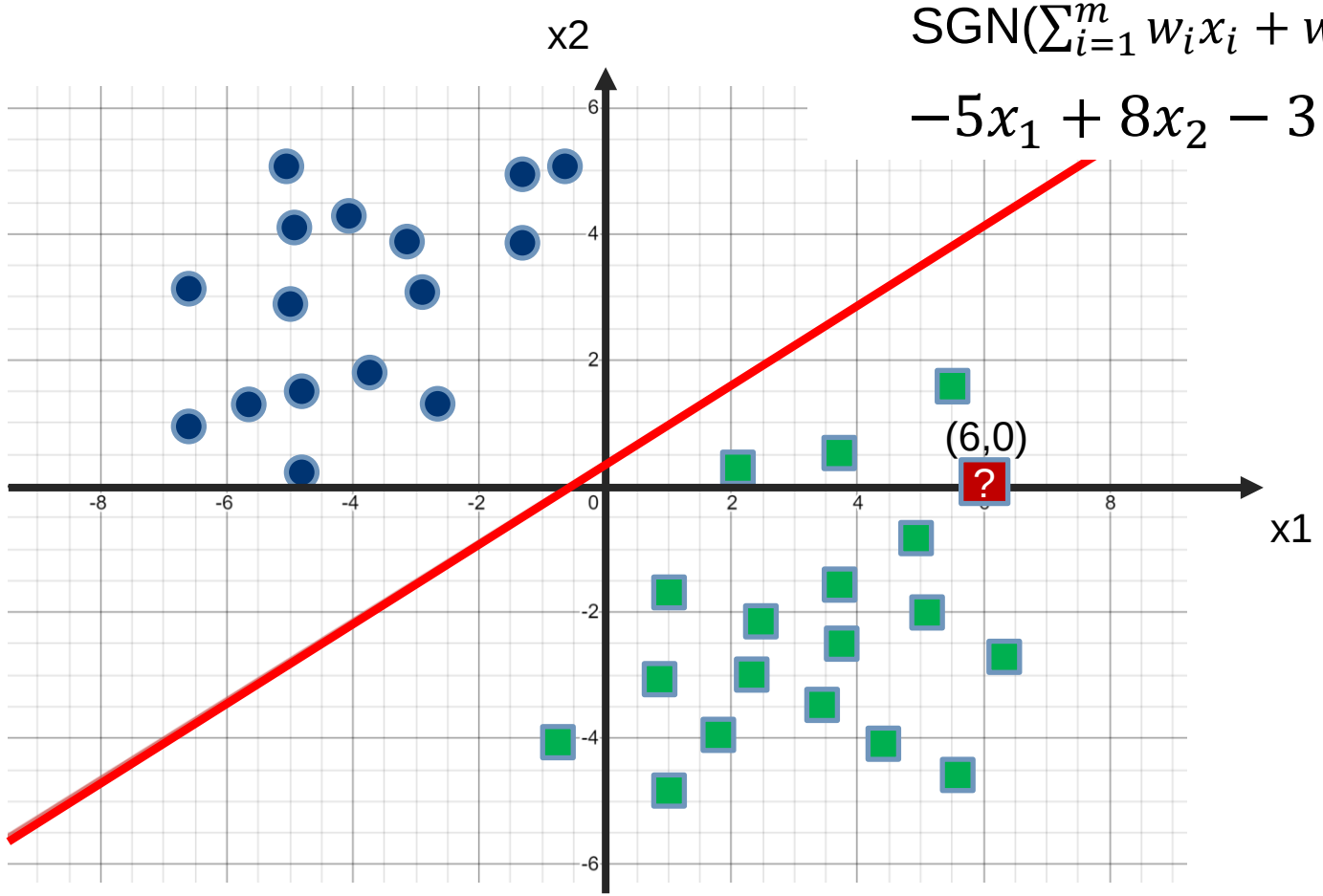
$(\quad w_1 = -5 \quad w_2 = +8)$ $\qquad Threshold = 3$

$(\quad x_1 = 2, \qquad x_2 = 4)$

$$(-5,\ 8)\begin{pmatrix}2\\4\end{pmatrix} = -5 \times 2 + 8 \times 4 - 3 > 0$$

Classify as POSITIVE

מסווג לינארי – משוואת הסיווג



$$\mathrm{SGN}\left(\sum_{i=1}^{m} w_i x_i + w_0\right)$$

$$-5x_1 + 8x_2 - 3 = 0$$

מסווג לינארי

Hyperplane equation : $-5x_1 + 8x_2 - 3 = 0$

Vector to classify : $(x_1 = 6, x_2 = 0)$

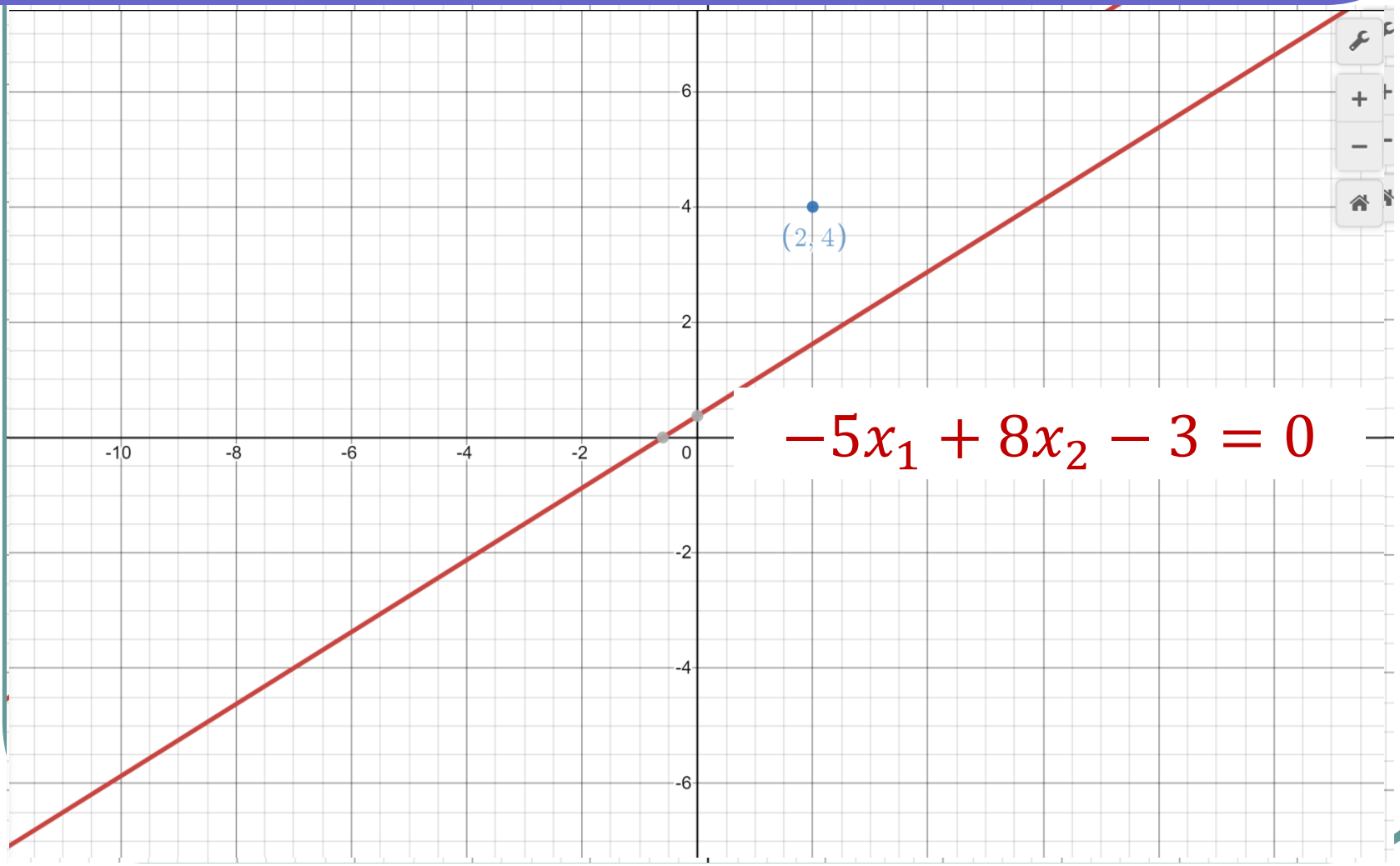
$(\quad w_1 = -5 \quad w_2 = +8)$ $\qquad Threshold = 3$

$(\quad x_1 = 6, \qquad x_2 = 0)$

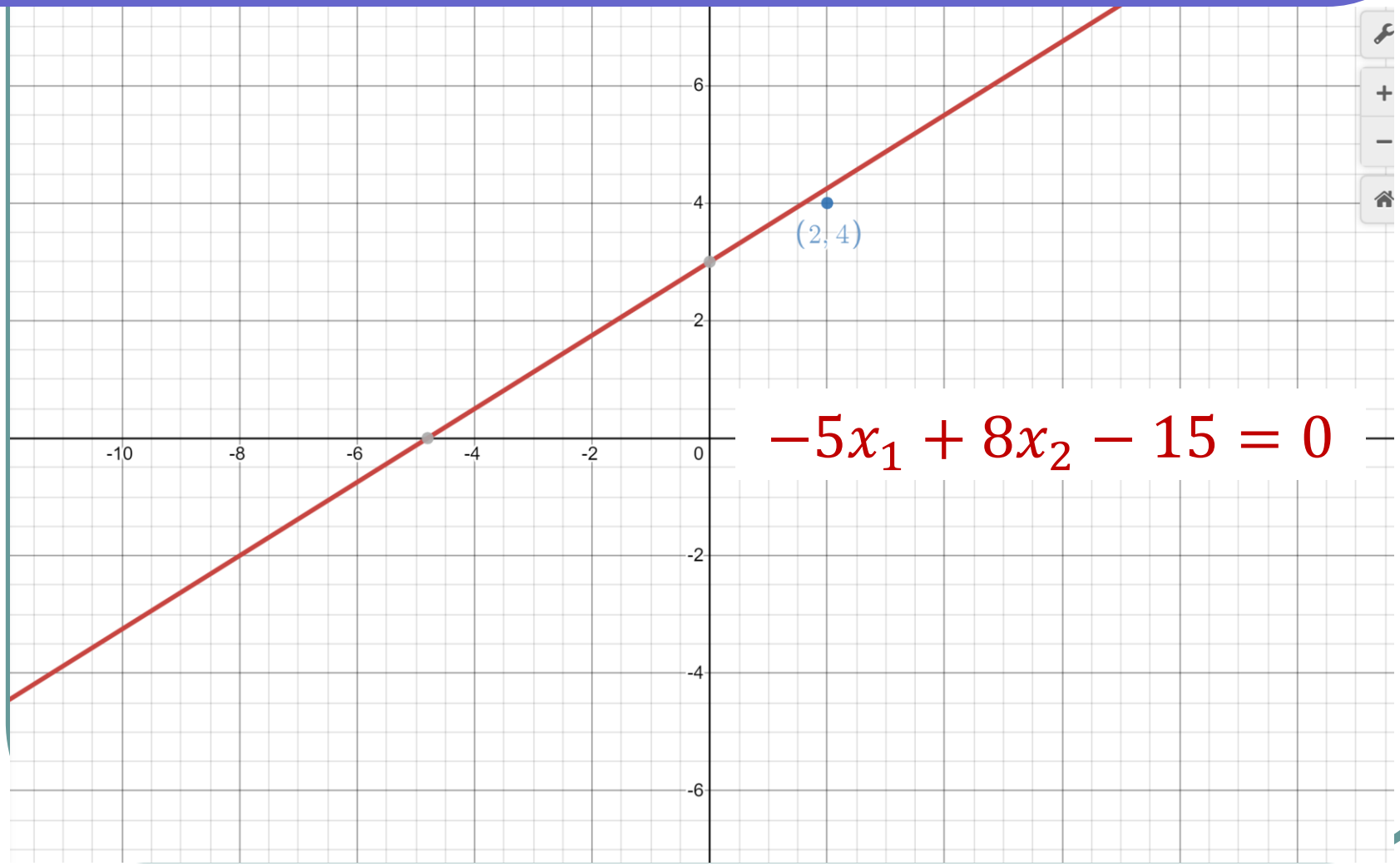
$$(-5,\ 8)\begin{pmatrix}6\\0\end{pmatrix} = -5 \times 6 + 8 \times 0 - 3 < 0$$

Classify as NEGATIVE

Reminder: Threshold effect



Reminder: Threshold effect



$$-5x_1 + 8x_2 - 15 = 0$$

רשתות עצביות Neural networks

- רשת עצבית (neural network) טבעית מורכבת מקבוצה או מקבוצות של תאי עצב המחוברים ביניהם. תא עצב אחד יכול להיות מחובר לתאי עצב רבים אחרים.

- רשת עצבית מלאכותית (Artificial Neural network) - מודל מתמטי חישובי המנסה לבצע הדמיה של תהליכים מוחיים, ושל תהליכים המתרחשים ברשת עצבית טבעית.

30

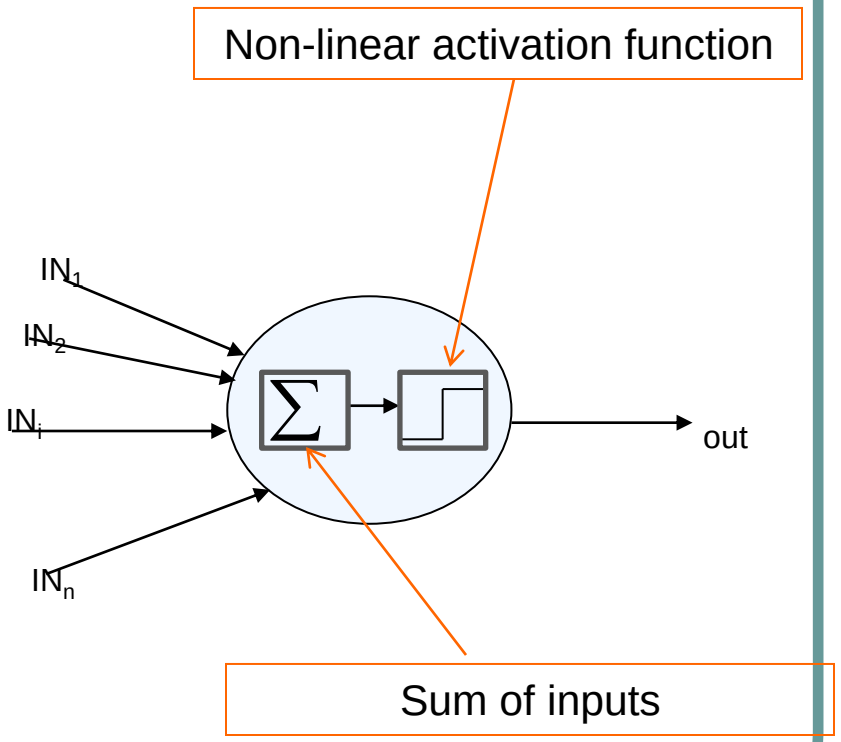
The McCulloch-Pitts Neuron

$$out = \mathrm{sgn}(\sum_{i=1}^{N} in_i - \theta)$$

θ is the neuron's activation threshold.

$$\text{out} = 1 \quad \text{if} \quad \sum_{i=1}^{n} \text{in}_i \geq \theta$$

$$\text{out} = 0 \quad \text{if} \quad \sum_{i=1}^{n} \text{in}_i \leq \theta$$

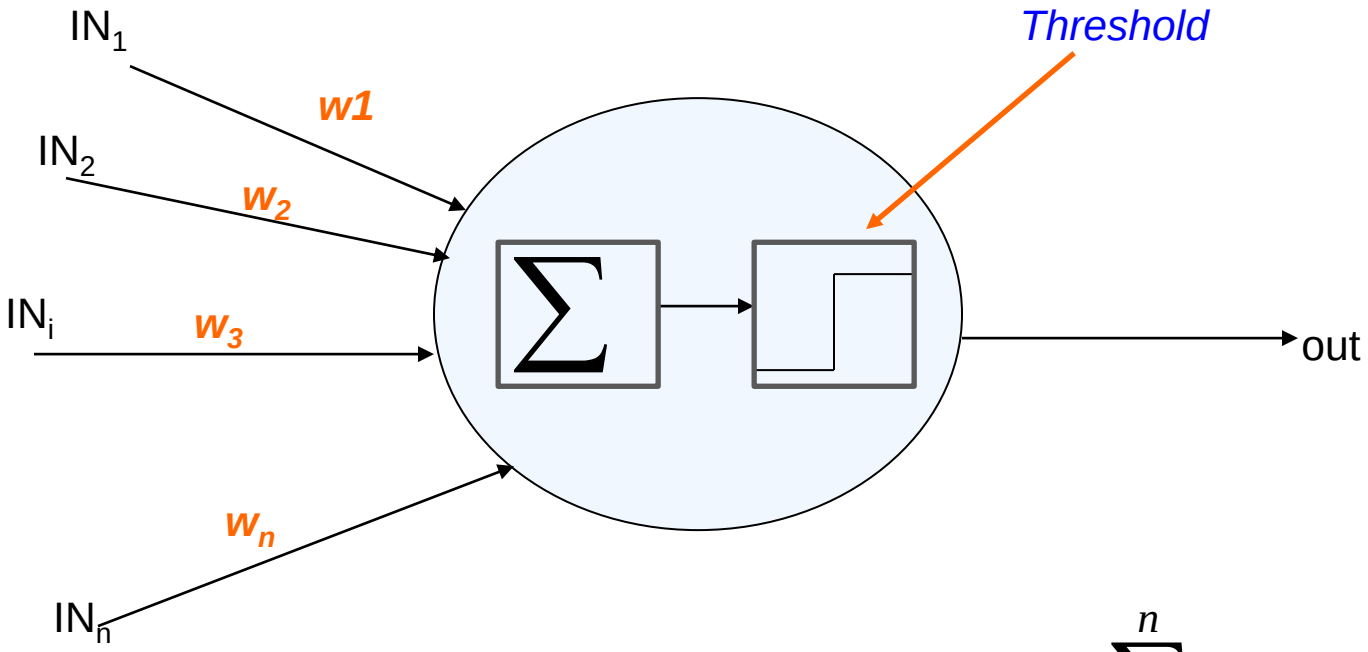


מודל פשוט מאוד של התנהגות נוירון

31

Basic Neuron

- aka adaline (Adaptive Linear Element)

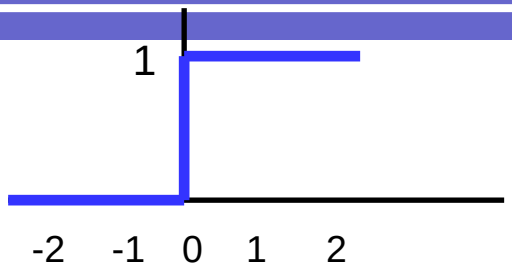


$$out = threshold(\sum_{i=1}^{n} in_i w_{ij}, \theta)$$

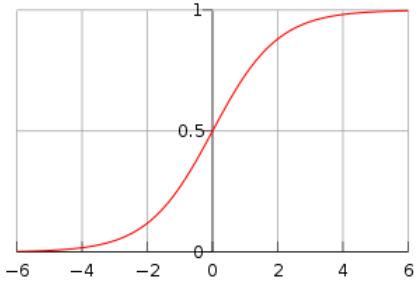
32

Activation functions examples

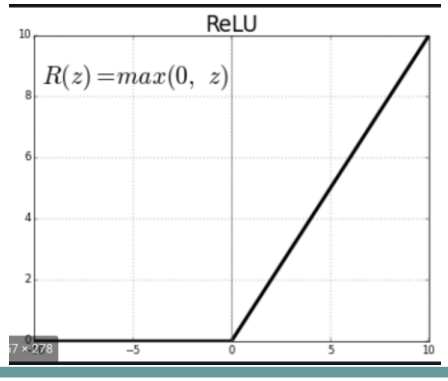
$$\text{sgn}(x) = \begin{cases} 1 \text{ if } x \geq 0 \\ 0 \text{ if } x < 0 \end{cases}$$



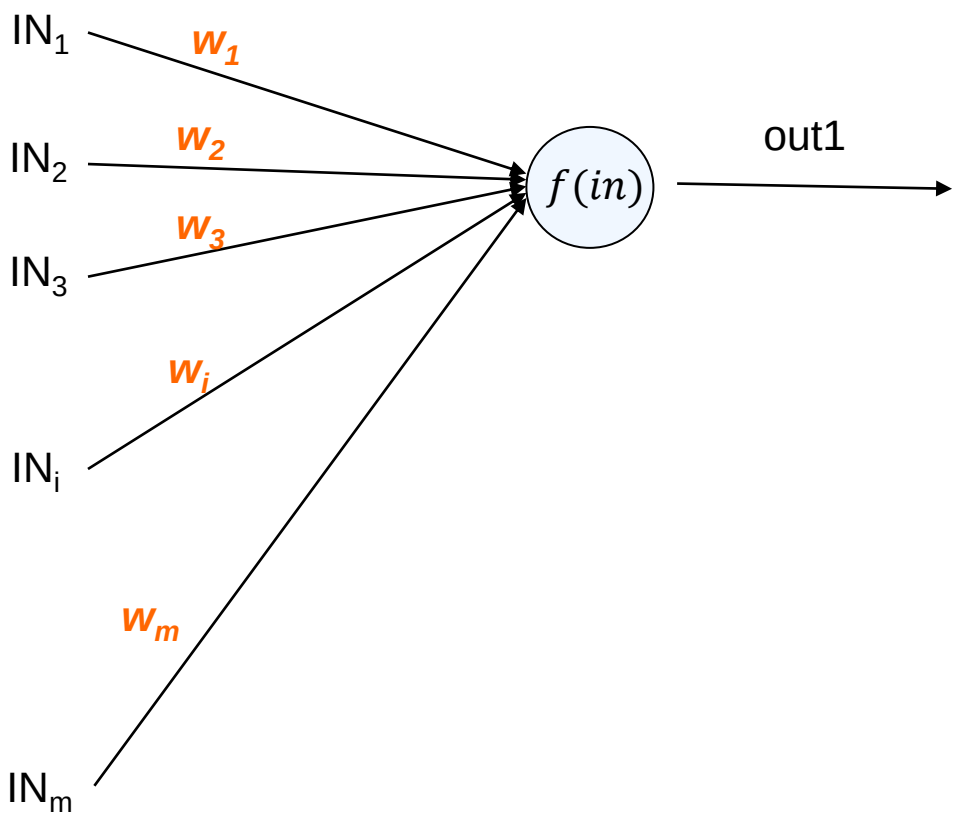
$$sigmoid(x) = \frac{1}{1 + e^{-x}}$$



$$RELU(x) = \max(0, x)$$

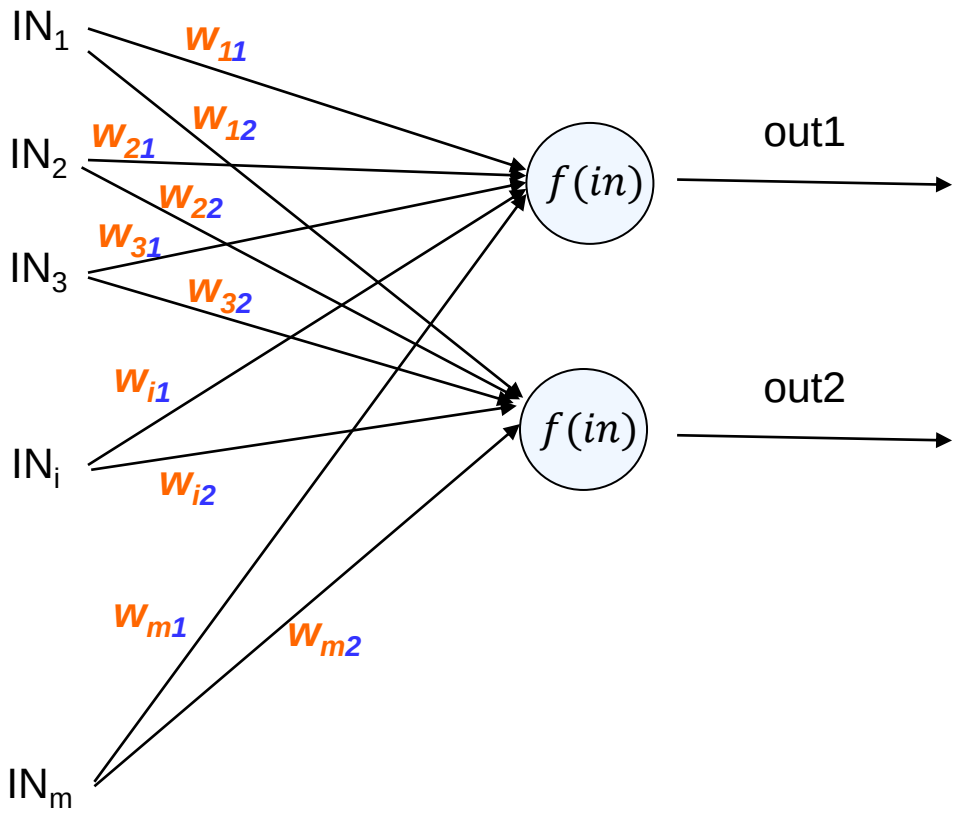


Basic Network



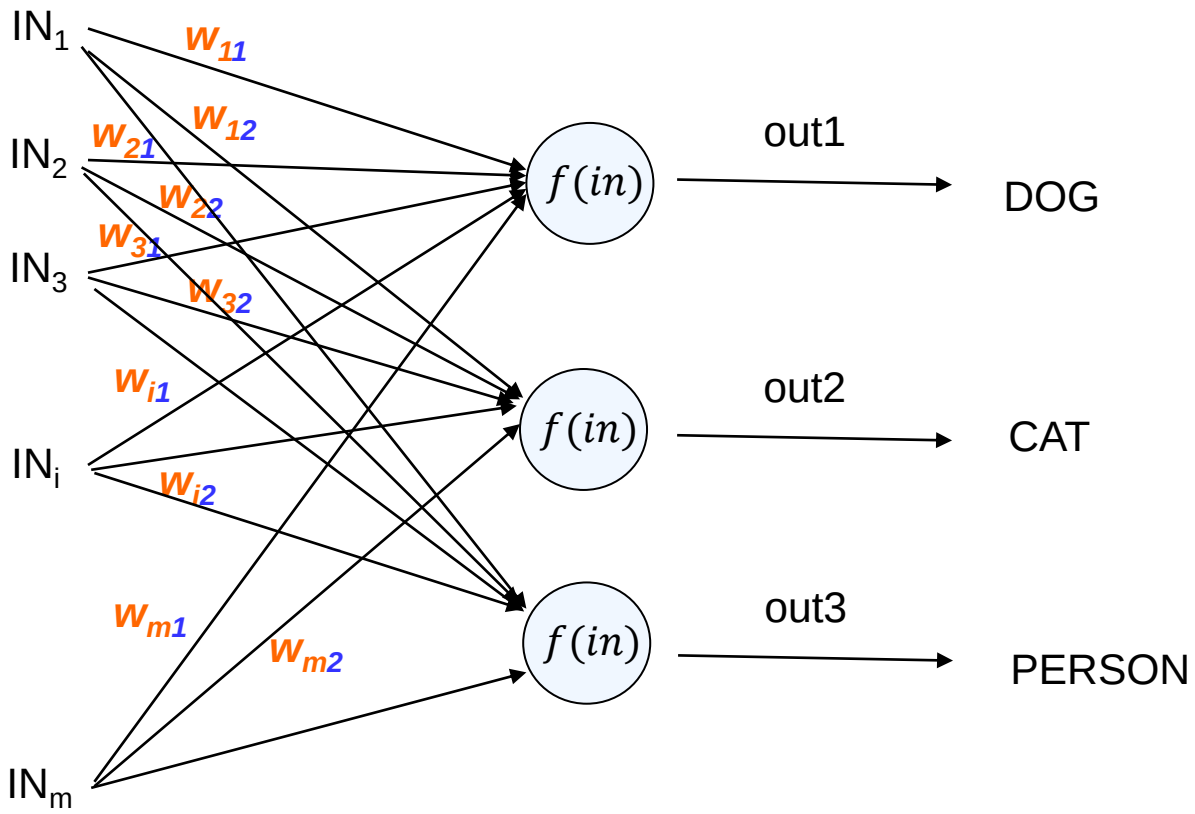
34

Basic Network



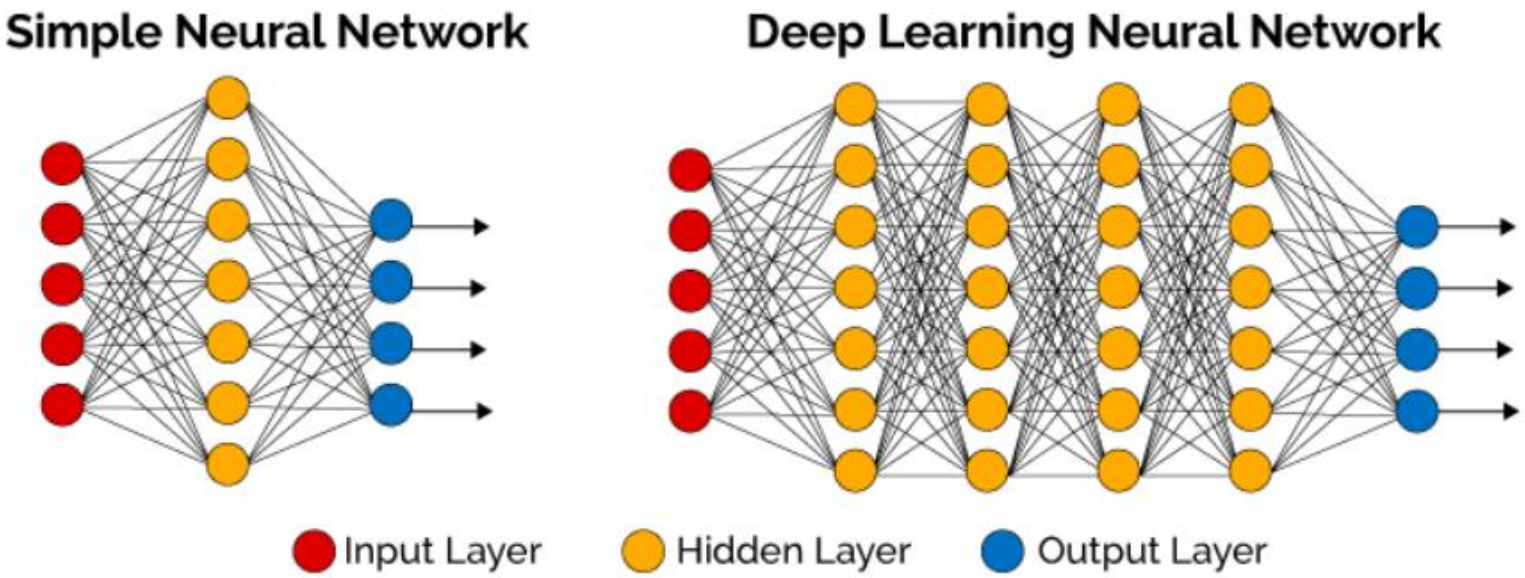
35

Basic Network



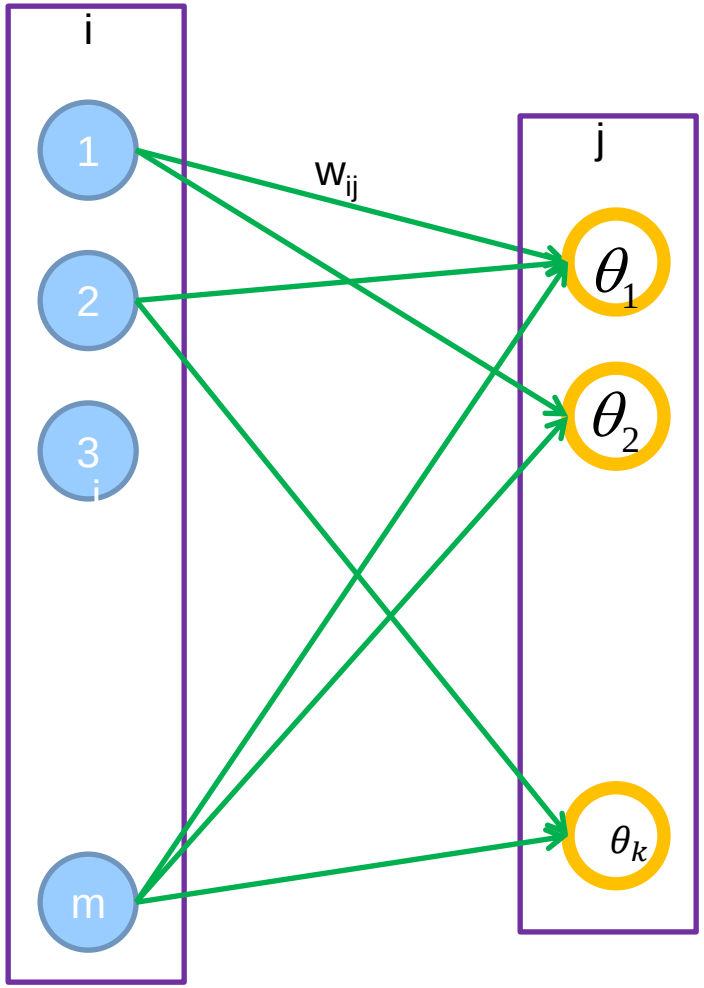
36

Neural Network רשת נוירונים



https://becominghuman.ai/deep-learning-made-easy-with-deep-cognition-403fbe445351

Perceptron



We can connect any number of McCulloch-Pitts neurons together in any way.

An arrangement of one input layer and one output layer of McCulloch-Pitts neurons is known as a ***Perceptron.***

$$out_j = \text{sgn}(\sum_{i=1}^{m} in_i w_{ij} - \theta)$$

צורה נוחה יותר לכתיבה מתמטית

$$out_j = \text{sgn}(\sum_{i=1}^{m} in_i w_{ij} - \theta)$$

צורה נוחה יותר לכתיבה מתמטית

$$out_j = \text{sgn}(\sum_{i=1}^{m} in_i w_{ij} - \theta)$$

$$= \text{sgn}(\boldsymbol{in_1 w_{1j} + in_2 w_{2j} + in_3 w_{3j} + \ldots + in_m w_{mj} - \theta})$$

$$= \text{sgn}(\boldsymbol{in_0 w_{0j} + in_1 w_{1j} + in_2 w_{2j} + in_3 w_{3j} + \ldots + in_m w_{mj}})$$

צורה נוחה יותר לכתיבה מתמטית

$$out_j = \text{sgn}(\sum_{i=1}^{m} in_i w_{ij} - \theta)$$

$$= \text{sgn}(\boldsymbol{in_1 w_{1j} + in_2 w_{2j} + in_3 w_{3j} + \ldots + in_m w_{mj} - \theta})$$

$$= \text{sgn}(\boldsymbol{in_0 w_{0j} + in_1 w_{1j} + in_2 w_{2j} + in_3 w_{3j} + \ldots + in_m w_{mj}})$$

$$= \text{sgn}(\sum_{i=0}^{m} in_i w_{ij})$$

בעצם מוסיפים נוירון "דמי" ע"י
עבורו $in_0 = 1$

בהינתן ווקטור לסיווג ופרספטרון

- בהינתן ווקטור לסיווג (x) ובהינתן סט משקולות (w)

$$(w_0 \quad w_1 \quad w_2 \quad \dots \quad \dots \quad w_d)\begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ \vdots \\ \\ x_d \end{pmatrix} = \sum_{i=0}^{d} w_i x_i = W^T x$$

- נסווג את הווקטור עפ"י תוצאת המכפלה הפנימית בין X ו-W והשוואה לסף.

מסווג לינארי

Hyperplane equation : $-5x_1 + 8x_2 - 3 = 0$

Vector to classify : $(x_1 = 2, x_2 = 4)$

$(w_0 = -3, \quad w_1 = -5 \quad w_2 = +8)$

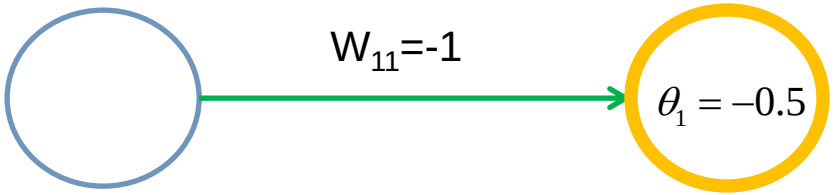
$(x_1 = 2, x_2 = 4) \longrightarrow (x_0 = 1, x_1 = 2, x_2 = 4)$

$$(-3, \quad -5, \quad 8) \begin{pmatrix} 1 \\ 2 \\ 8 \end{pmatrix} = -3 \times 1 - 5 \times 2 + 8 \times 8 = 51 > 0$$

Classify as POSITIVE

NOT

NOT	
X	Y
0	1
1	0



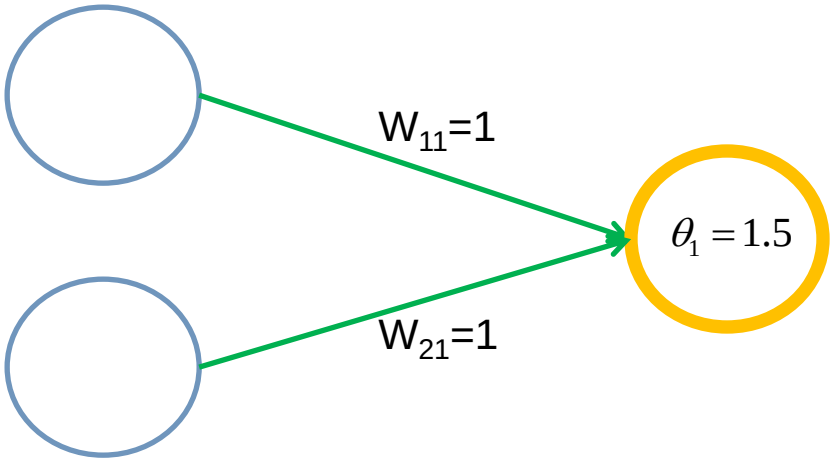
$$out_j = \text{sgn}(\sum_{i=1}^{n} in_i w_{ij} - \theta)$$

$$out(0) = \text{sgn}(0 \times -1 - (-0.5)) = \text{sgn}(0.5) = 1$$

$$out(1) = \text{sgn}(1 \times -1 - (-0.5)) = \text{sgn}(-0.5) = 0$$

AND

AND		
X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1



$$out_j = \mathrm{sgn}(\sum_{i=1}^{n} in_i w_{ij} - \theta)$$

$$out(0,0) = \mathrm{sgn}(0\times1+0\times1-1.5) = \mathrm{sgn}(-1.5) = 0$$

$$out(0,1) = \mathrm{sgn}(0\times1+1\times1-1.5) = \mathrm{sgn}(-0.5) = 0$$

$$out(1,0) = \mathrm{sgn}(1\times1+0\times1-1.5) = \mathrm{sgn}(-0.5) = 0$$

$$out(1,1) = \mathrm{sgn}(1\times1+1\times1-1.5) = \mathrm{sgn}(0.5) = 1$$

XOR ???

XOR		
X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0

פתרון AND

AND		
X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1

$$out_j = \mathrm{sgn}(\sum_{i=1}^{n} in_i w_{ij} - \theta) = \mathrm{sgn}(in_1 w_{11} + in_2 w_{21} - \theta)$$

$$out(0,0) = 0 \Rightarrow 0 \times w_{11} + 0 \times w_{21} - \theta < 0 \Rightarrow \theta > 0$$

$$out(0,1) = 0 \Rightarrow 0 \times w_{11} + 1 \times w_{21} - \theta < 0 \Rightarrow w_{21} < \theta$$

$$out(1,0) = 0 \Rightarrow 1 \times w_{11} + 0 \times w_{21} - \theta < 0 \Rightarrow w_{11} < \theta$$

$$out(1,1) = 1 \Rightarrow 1 \times w_{11} + 1 \times w_{21} - \theta > 0 \Rightarrow w_{21} + w_{11} \geq \theta$$

Infinite number of solutions
e.g. $\theta = 1.5$, $w_{11} = 1$, $w_{21} = 1$

פתרון XOR

XOR		
X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0

$$out_j = \mathrm{sgn}(\sum_{i=1}^{n} in_i w_{ij} - \theta) = \mathrm{sgn}(in_1 w_{11} + in_2 w_{21} - \theta)$$

$$out(0,0) = 0 \Rightarrow 0 \times w_{11} + 0 \times w_{21} - \theta < 0 \Rightarrow \theta > 0$$

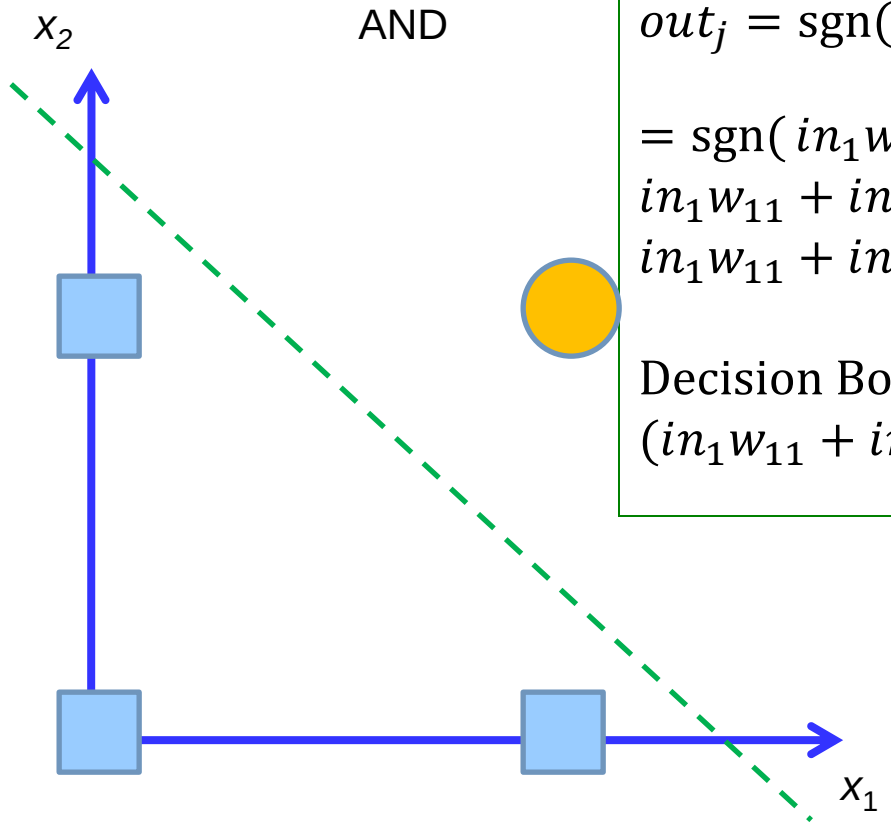
$$out(0,1) = 1 \Rightarrow 0 \times w_{11} + 1 \times w_{21} - \theta \geq 0 \Rightarrow w_{21} \geq \theta$$

$$out(1,0) = 1 \Rightarrow 1 \times w_{11} + 0 \times w_{21} - \theta \geq 0 \Rightarrow w_{11} \geq \theta$$

$$out(1,1) = 0 \Rightarrow 1 \times w_{11} + 1 \times w_{21} - \theta < 0 \Rightarrow w_{21} + w_{11} < \theta$$

לא קיים פתרון המתאים למשוואה שבחרנו !!!!

Decision Boundary



$$out_j = \text{sgn}(\sum_{i=1}^{n} in_i w_{ij} - \theta)$$

$$= \text{sgn}(in_1 w_{11} + in_2 w_{21} - \theta)$$

$$in_1 w_{11} + in_2 w_{21} - \theta \geq 0 \Rightarrow out = 1$$

$$in_1 w_{11} + in_2 w_{21} - \theta < 0 \Rightarrow out = 0$$

Decision Boundry is when

$$(in_1 w_{11} + in_2 w_{21} - \theta = 0)$$

Single layer generates a linear decision boundary

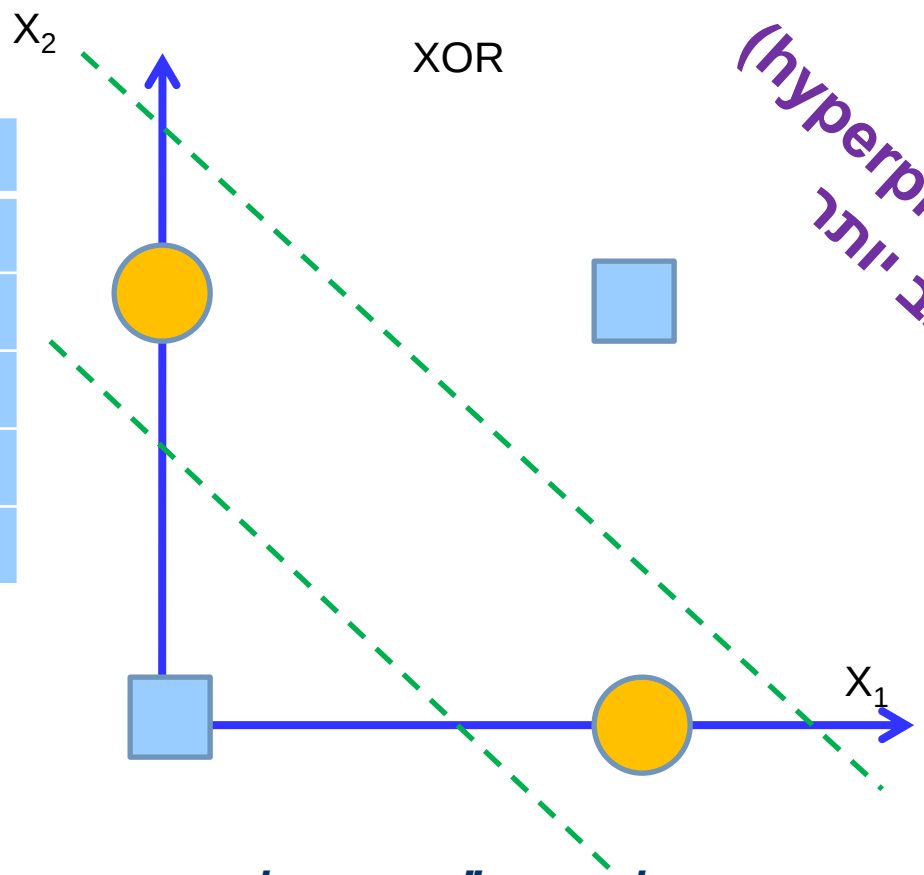
Decision Boundary

$$(w_0 \quad w_1 \quad w_2 \quad \dots \quad \dots \quad w_d) \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ \vdots \\ x_d \end{pmatrix} = \sum_{i=0}^{d} w_i x_i = W^T x$$

Single layer generates a linear decision boundary

Decision Boundary - XOR

XOR		
X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0



נדרש "על-מישור (hyperplane)
הפרדה" מורכב יותר

בעיה שאינה ניתנת לפתרון ע"י הפרדה לינארית

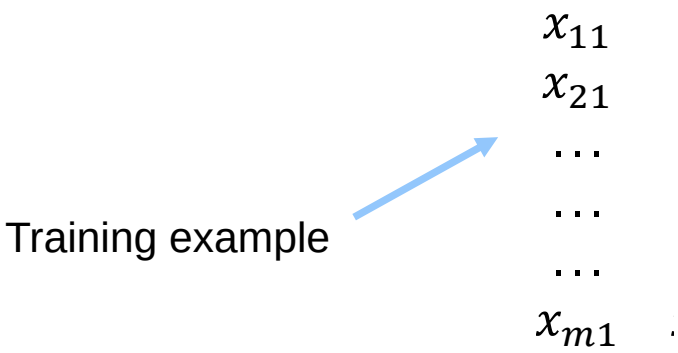
Non Linearly Separable vs. Linearly separable

אז מה יש לנו עד כה...

- מודל מתמטי שיודע לקבל קבוצת קלטים ולהוציא פלט/ים
- איך זה עוזר לנו ???
- איך מחשבים את הקשרים המתמטיים???

ייצוג מתמטי

- נתונים לנו k ווקטורים (דוגמאות) - $\boldsymbol{x_i} \in R^m$
-



ייצוג מתמטי

- נתונים לנו k ווקטורים (דוגמאות) - $\boldsymbol{x_i} \in R^m$
-

נוירון דמי

x_{01}
x_{11}
x_{21}
$\dots$
$\dots$
$\dots$
x_{m1}

Training example

ייצוג מתמטי

- נתונים לנו k ווקטורים (דוגמאות) - $\boldsymbol{x_i} \in R^m$
- $$\begin{bmatrix} x_{01} & x_{02} & x_{03} & \cdots & \cdots & \cdots & x_{0k} \\ x_{11} & x_{12} & x_{13} & & & & x_{1k} \\ x_{21} & x_{22} & x_{23} & & & & x_{2k} \\ \cdots & \cdots & \cdots & & & & \\ \cdots & \cdots & \cdots & & & & \\ \cdots & \cdots & \cdots & & & & \\ x_{m1} & x_{m2} & x_{m3} & & & & x_{mk} \end{bmatrix}$$

כל עמודה במטריצה מייצגת דוגמא בודדת

סה"כ K דוגמאות

כל דוגמא במימד m

ייצוג מתמטי

- נתונים לנו k ווקטורים (דוגמאות) - $\boldsymbol{x_i} \in R^m$
- $$\begin{bmatrix} x_{01} & x_{02} & x_{03} & \cdots & \cdots & \cdots & x_{0k} \\ x_{11} & x_{12} & x_{13} & & & & x_{1k} \\ x_{21} & x_{22} & x_{23} & & & & x_{2k} \\ \cdots & \cdots & \cdots & & & & \\ \cdots & \cdots & \cdots & & & & \\ \cdots & \cdots & \cdots & & & & \\ x_{m1} & x_{m2} & x_{m3} & & & & x_{mk} \end{bmatrix}$$

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \\ \\ \\ y_k \end{bmatrix}$$

$$y_i \in \{0,1,2, \ldots \ldots \ldots \#ofclasses)$$

ייצוג מתמטי – סיווג בינארי

- אנו מחפשים קבוצת משקלים כך ש-

$$\begin{aligned}
&\operatorname{sgn}(w_0 x_{01} + w_1 x_{11} + w_2 x_{21} + \ldots\ldots + w_n x_{n1}) = y_1 \\
&\operatorname{sgn}(w_0 x_{02} + w_1 x_{12} + w_2 x_{22} + \ldots\ldots + w_n x_{n2}) = y_2 \\
&\ldots\ldots \\
&\ldots\ldots
\end{aligned}$$

$$\operatorname{sgn}\left(\left[w_0, w_1, w_2 \ldots\ldots w_n \right] \begin{bmatrix} x_{01} & x_{02} & x_{03} & \cdots & \cdots & \cdots & x_{0k} \\ x_{11} & x_{12} & x_{13} & \cdots & & & x_{1k} \\ x_{21} & x_{22} & x_{23} & \cdots & & & x_{2k} \\ \cdots & \cdots & \cdots & & & & \\ \cdots & \cdots & \cdots & & & & \\ \cdots & \cdots & \cdots & & & & \\ x_{n1} & x_{n2} & x_{n3} & & & & x_{nk} \end{bmatrix} \right) = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ . \\ . \\ . \\ y_k \end{bmatrix}$$

הרעיון הכללי

לא תמיד יש הפרדה לינארית –
האסטרטגיה שלנו: נחפש את הפתרון שיביא אותנו
למינימום טעות על כל הדוגמאות

- נתחיל מקבוצת משקולות רנדומאליות
- נגדיר פונקצית טעות
- נחשב את הטעות הכללית (סכימה של כל הטעויות על כל דוגמא)
- נעדכן בכל שלב את המשקולות ע"מ להקטין את הטעות הכללית

$$w_{ij} \rightarrow w_{ij} + \Delta w_{ij}$$

- נשתמש באלגוריתם gradient descent

ננסה להבין להיכן להתקדם

$Loss(\theta_1, \theta_2)=(\theta_1 - 1)^2+3(\theta_2 - 3)^2$

Initial Guess: (θ_1= 2, θ_2=5) ⇒ Loss= 13

$Loss(\theta_1 + \epsilon, \theta_2) = Loss(2.00001,5) = 13.00002$

Decrease θ_1

$Loss(\theta_1, \theta_2 + \varepsilon) = Loss(2,5.00001) = 13.0001$

Decrease θ_2

Try a new Guess: (θ_1= 1.5, θ_2=4) ⇒ Loss= 3.25

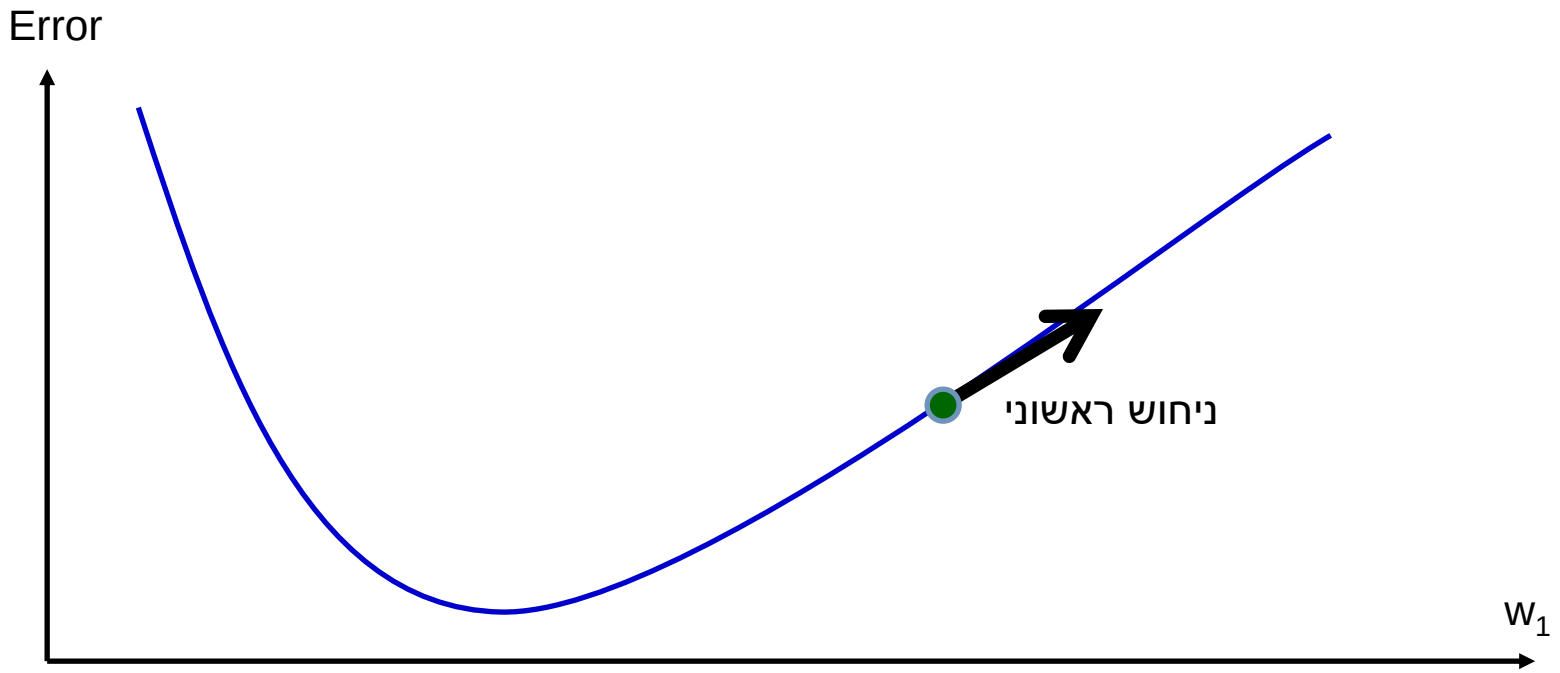
Gradient Descent – דרך מתוחכמת יותר

- **נניח שיש לנו את פונקציית הטעות E כפונציה של המשקל w:** $E = f(w)$
- **א':** כאשר $\frac{dE}{dw} = 0$ אנו נמצאים בנקודת מינימום או מקסימום
- **ב':** כאשר $\frac{dE}{dw} > 0$ אזי f(w) גדל ככל ש-w גדל
- **ג':** כאשר $\frac{dE}{dw} < 0$ אזי f(w) קטן ככל ש-w גדל

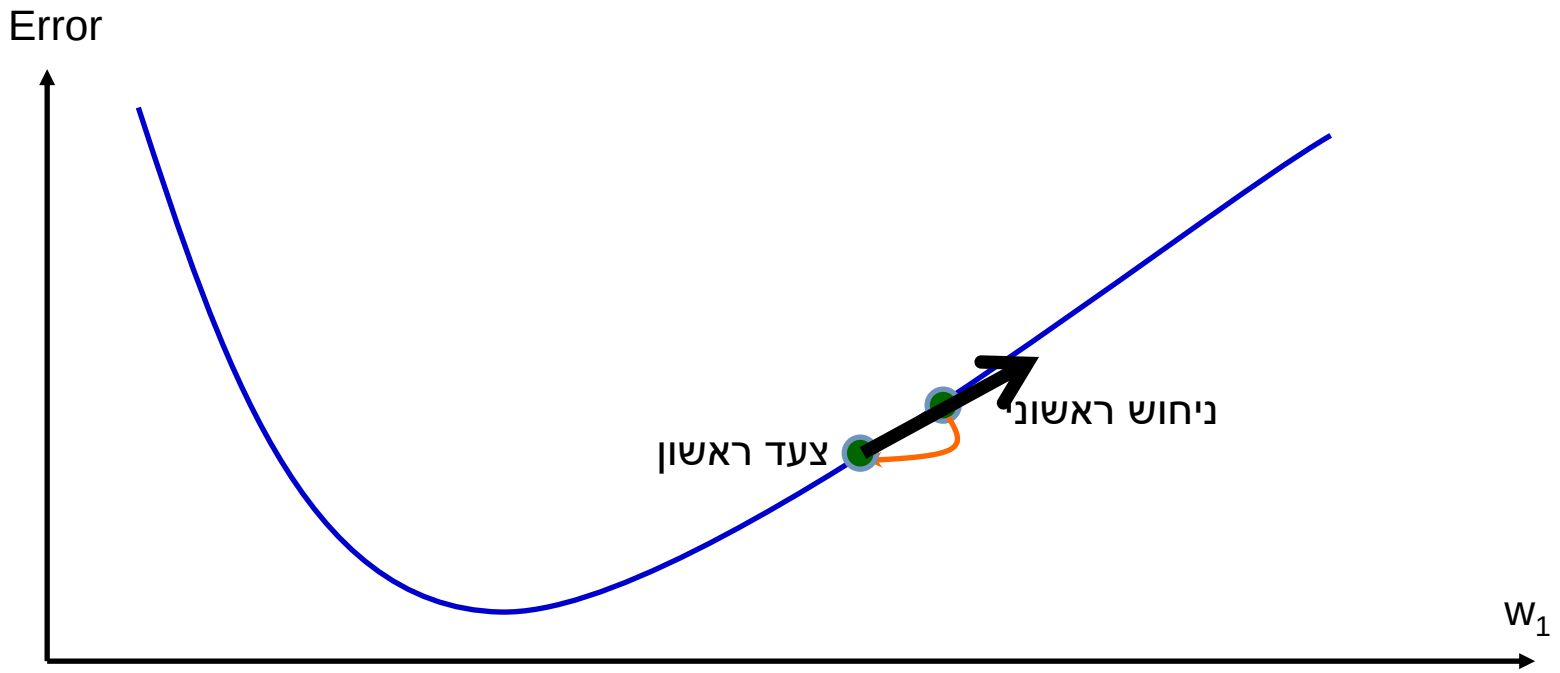
- E הינה פונקציית הטעות אותה נרצה להקטין, לכן:
  - עבור מקרה **ב'** – נרצה להקטין את w
  - עבור מקרה **ג'** – נרצה להגדיל את w
  - צריך ללכת בכיוון הפוך לעוצמת הגרדיאנט ←

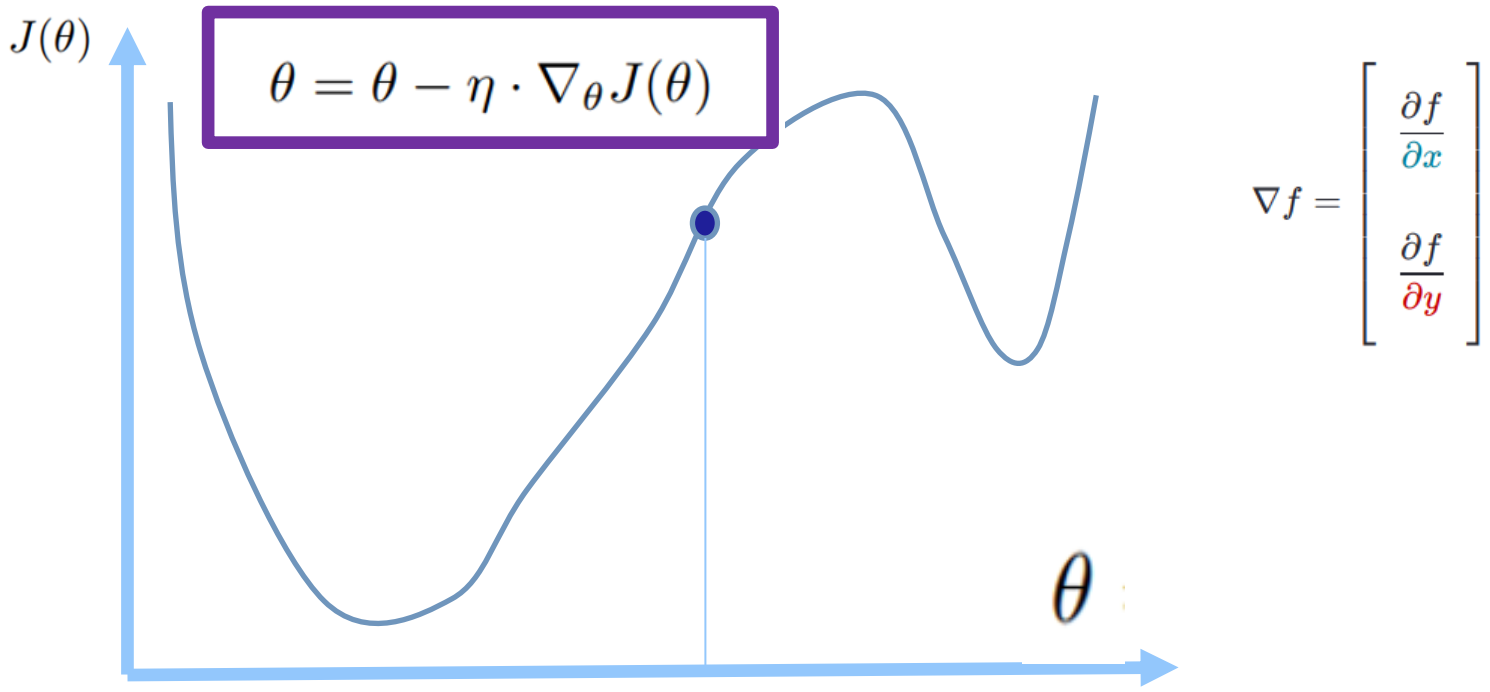
$$\Delta w = -\eta \frac{\partial E}{\partial w}$$

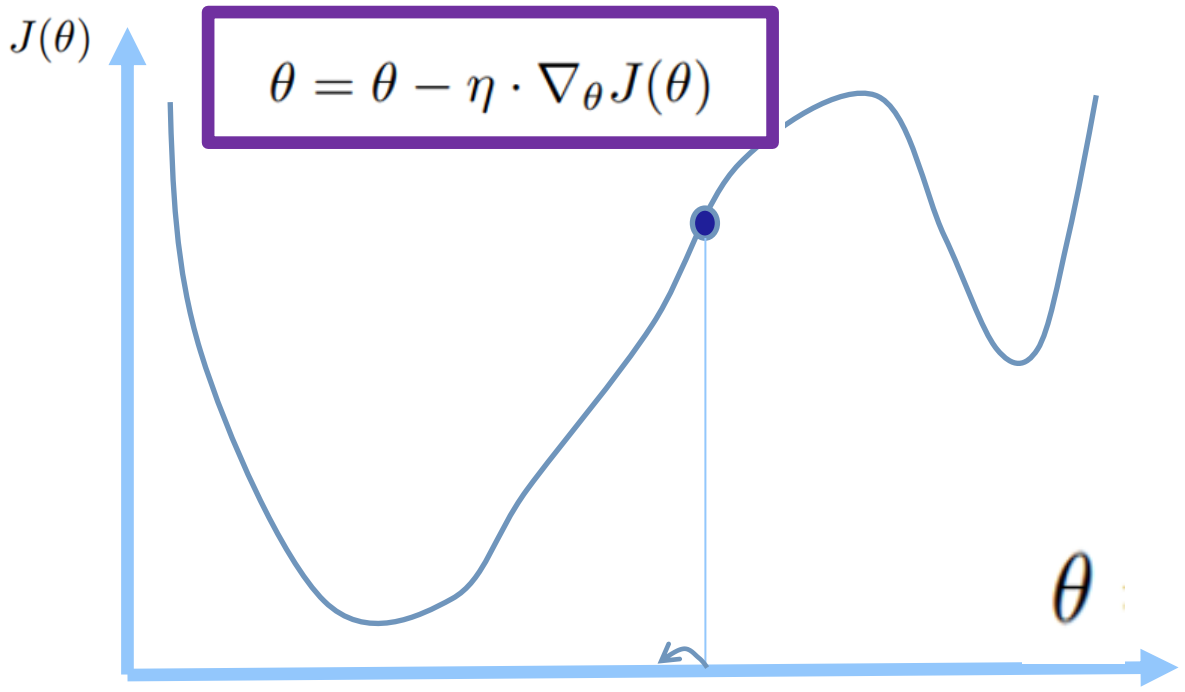
משתנה אחד – Gradient descent



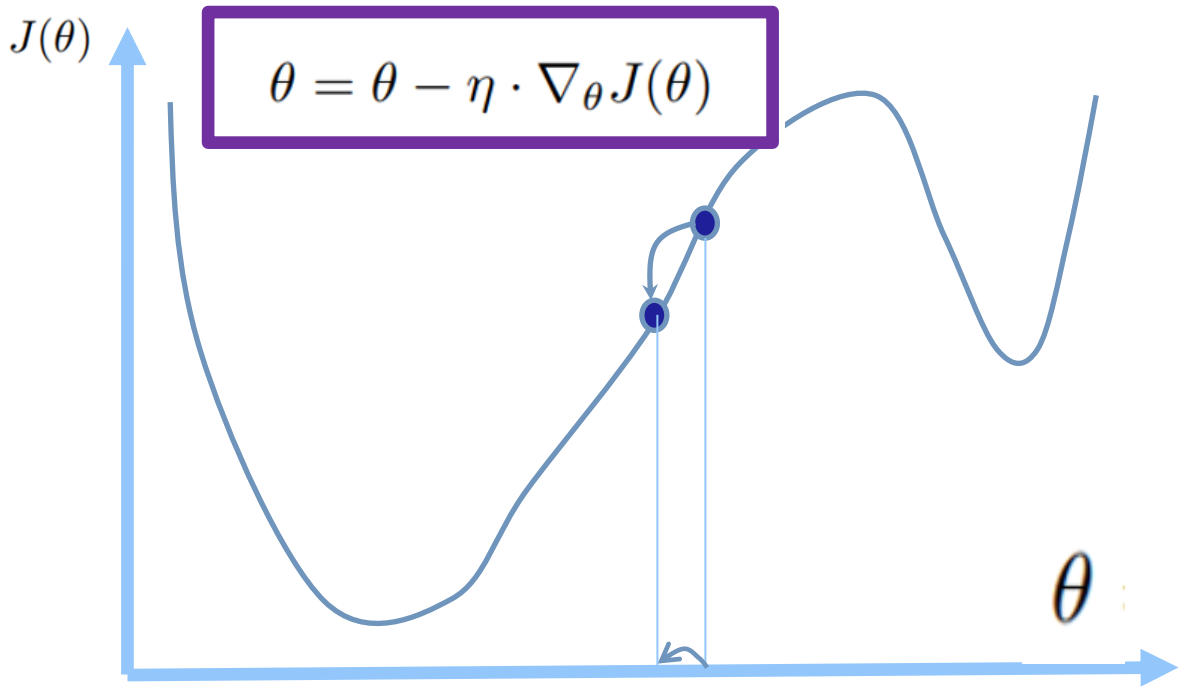
משתנה אחד – Gradient descent



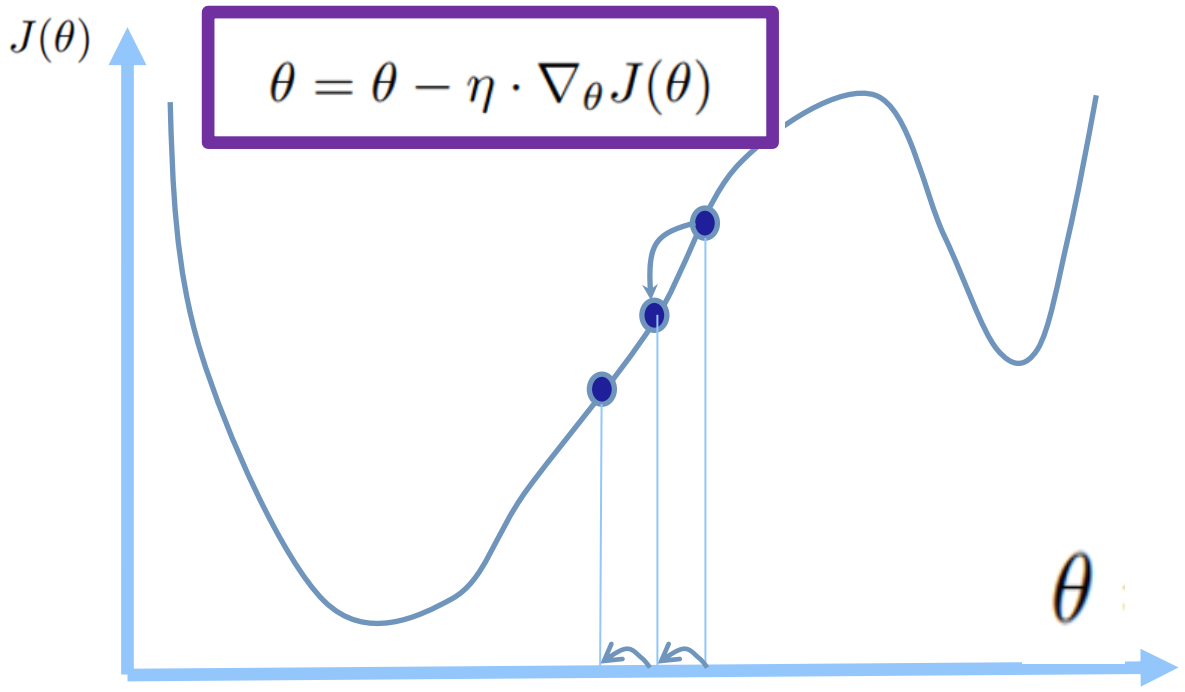

$J(\theta)$
$\theta = \theta - \eta \cdot \nabla_\theta J(\theta)$
θ
$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$



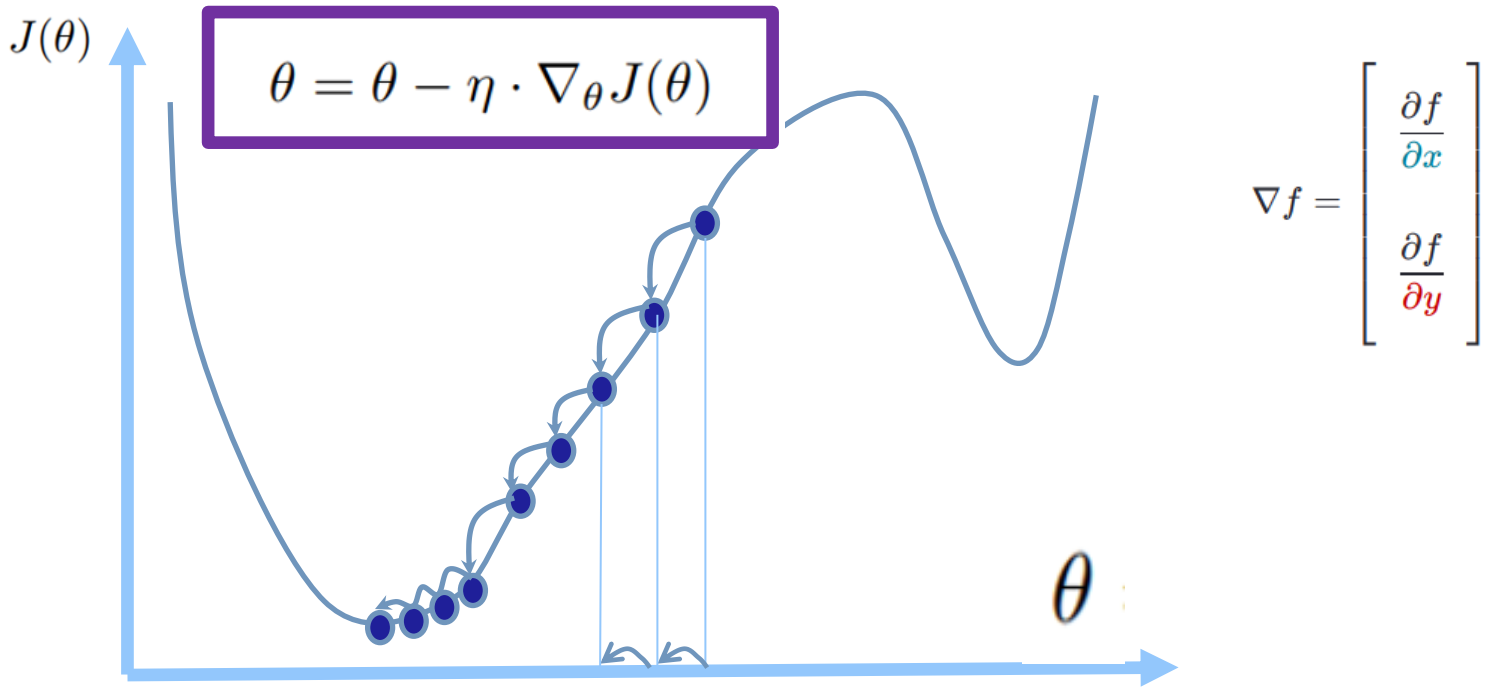
$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \\ \frac{\partial f}{\partial y} \end{bmatrix}$$



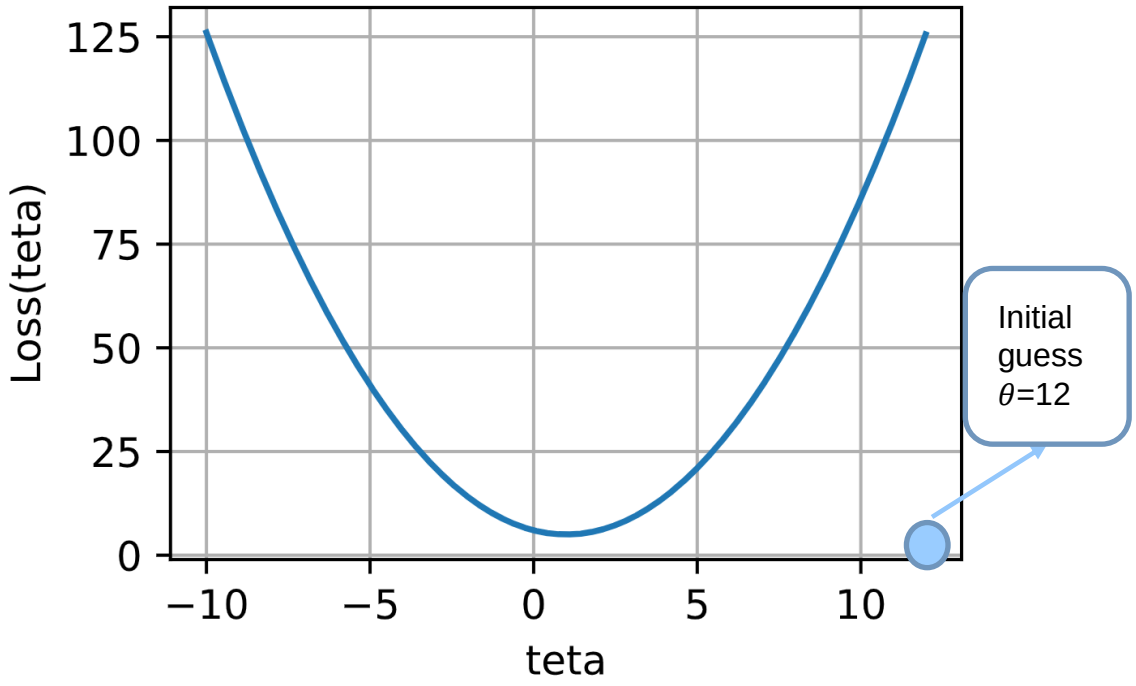
$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \\ \frac{\partial f}{\partial y} \end{bmatrix}$$



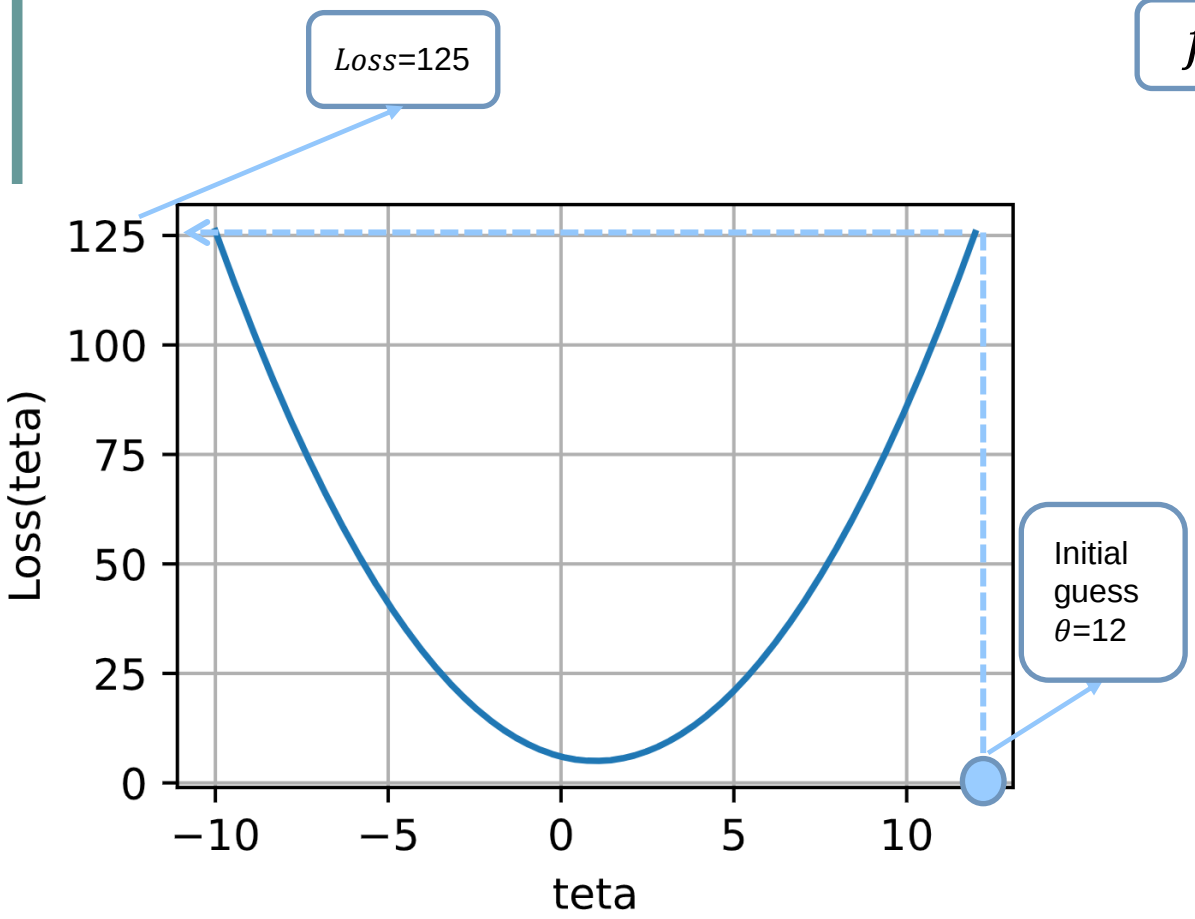
$$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \\ \frac{\partial f}{\partial y} \end{bmatrix}$$


$J(\theta)$
$\theta = \theta - \eta \cdot \nabla_\theta J(\theta)$
θ
$\nabla f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$

$$f(\theta) = (\theta - 1)^2 + 5$$

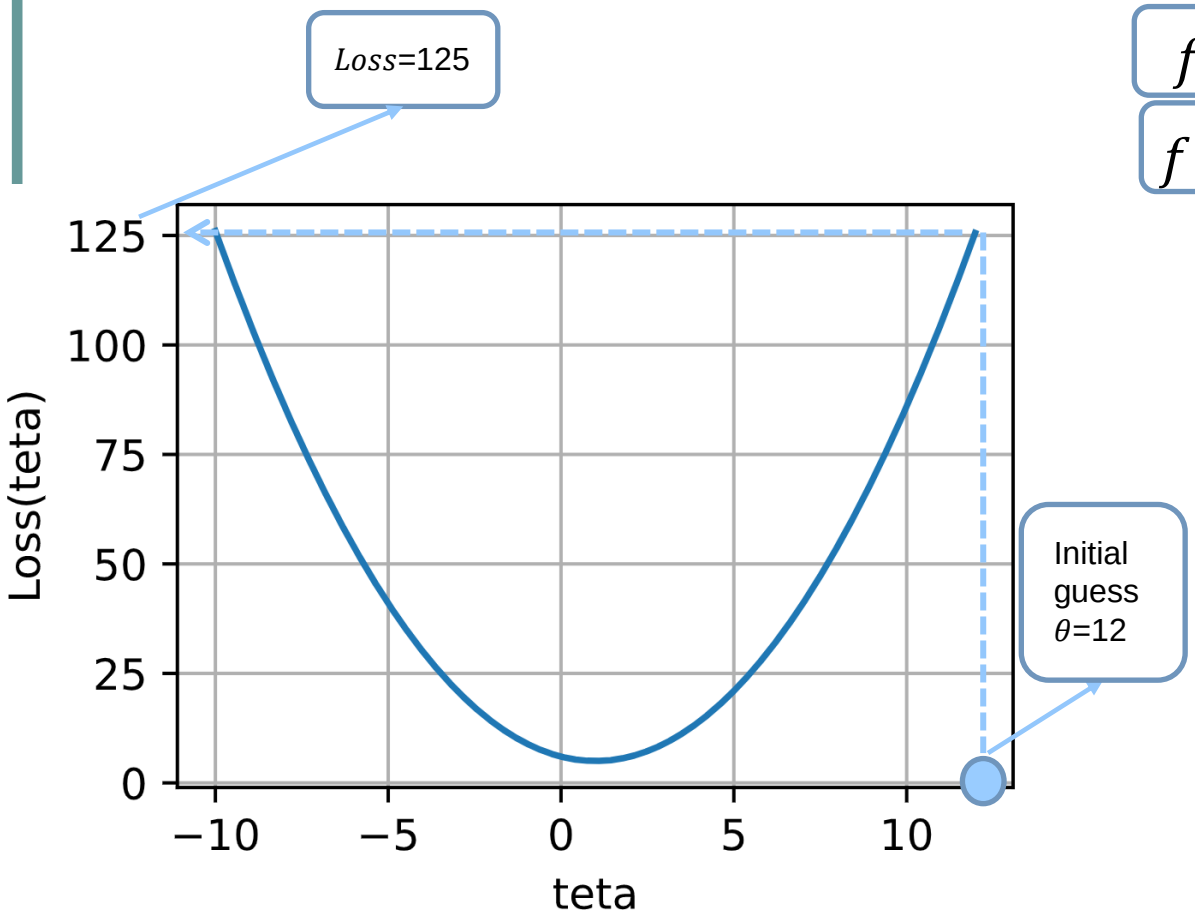


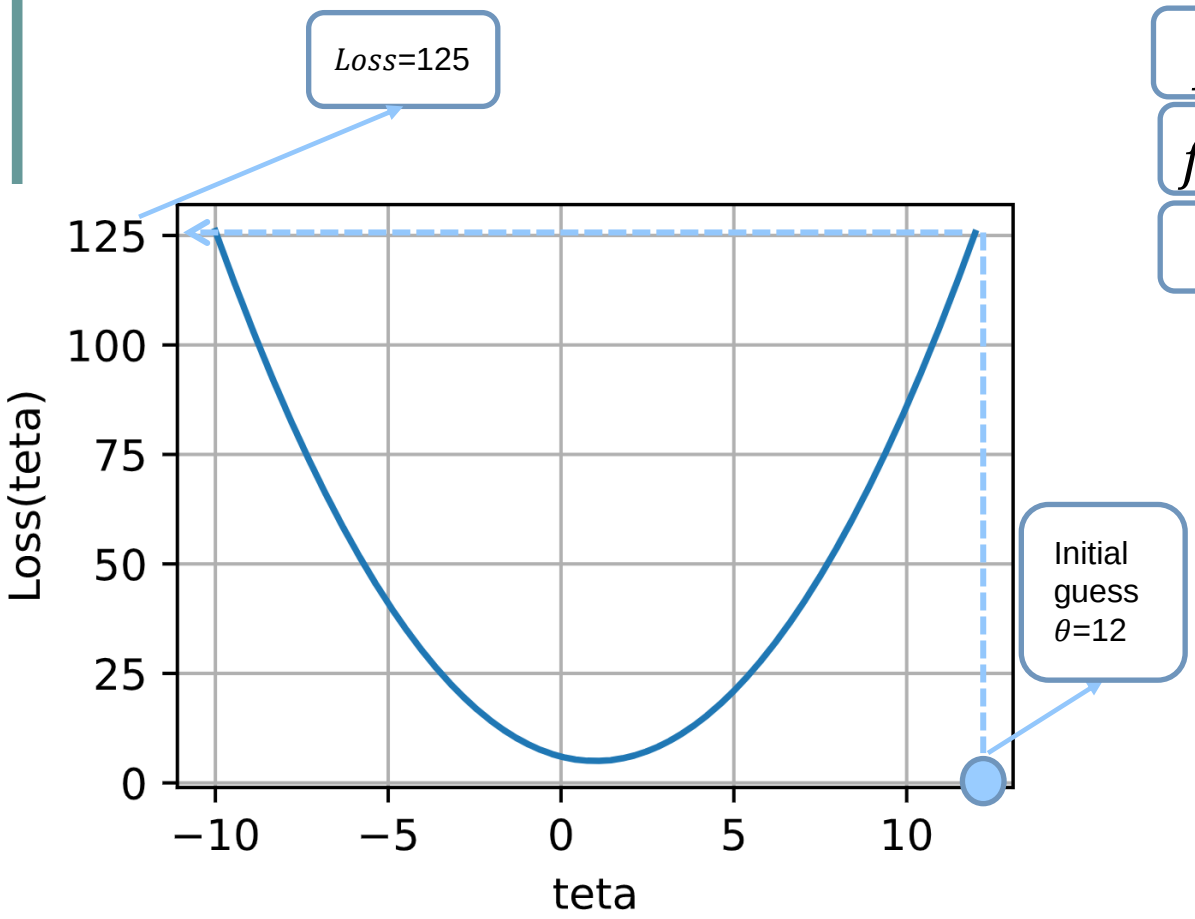
$f(\theta) = (\theta - 1)^2 + 5$



$f(\theta) = (\theta - 1)^2 + 5$

$f'(\theta) = 2(\theta - 1)$

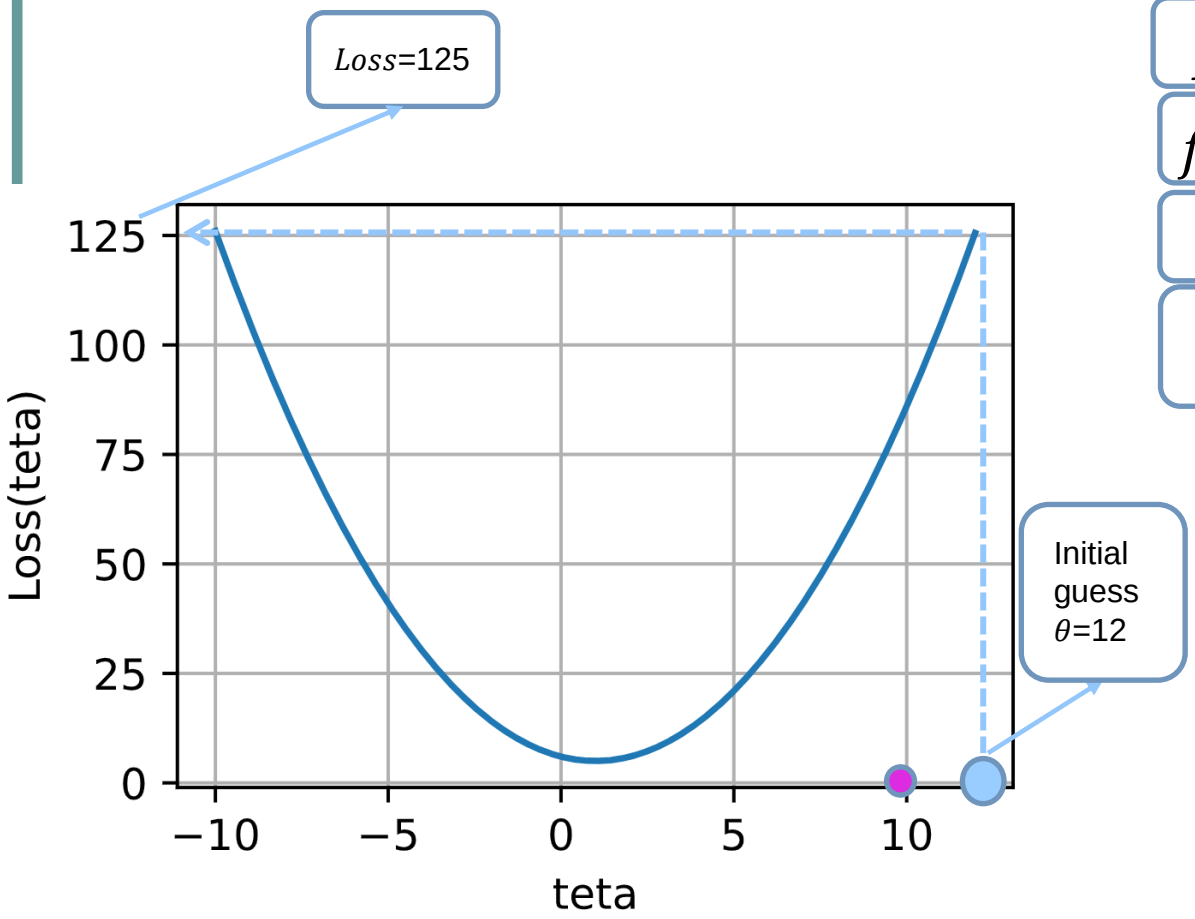




$$f(\theta) = (\theta - 1)^2 + 5$$

$$f'(\theta) = 2(\theta - 1)$$

$$f'(\theta = 12) = 2(12 - 1) = 22 > 0$$



$$f(\theta) = (\theta - 1)^2 + 5$$

$$f'(\theta) = 2(\theta - 1)$$

$$f'(\theta = 12) = 2(12 - 1) = 22 > 0$$

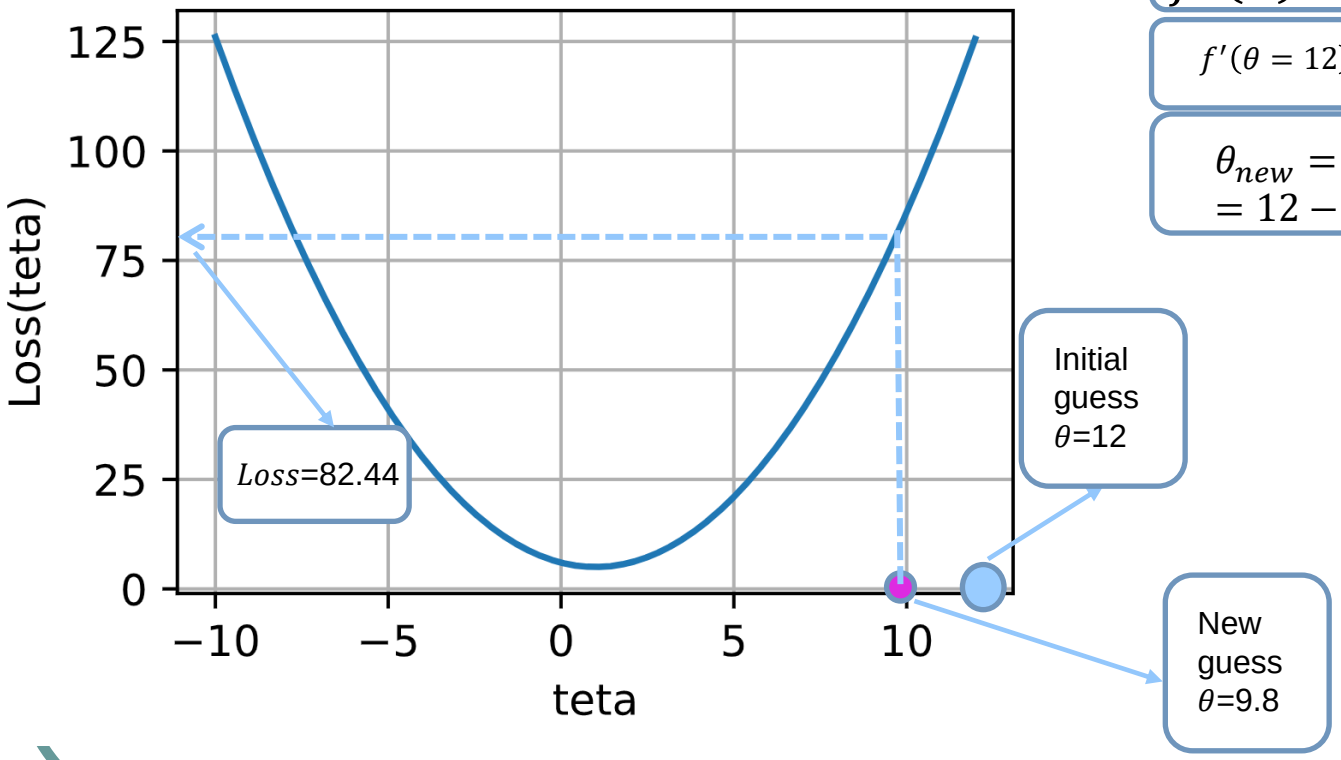
$$\theta_{new} = \theta_{old} - \eta f'(\theta = 12)$$
$$= 12 - 0.1 \times 22 = 9.8$$

$$f(\theta) = (\theta - 1)^2 + 5$$

$$f'(\theta) = 2(\theta - 1)$$

$$f'(\theta = 12) = 2(12 - 1) = 22 > 0$$

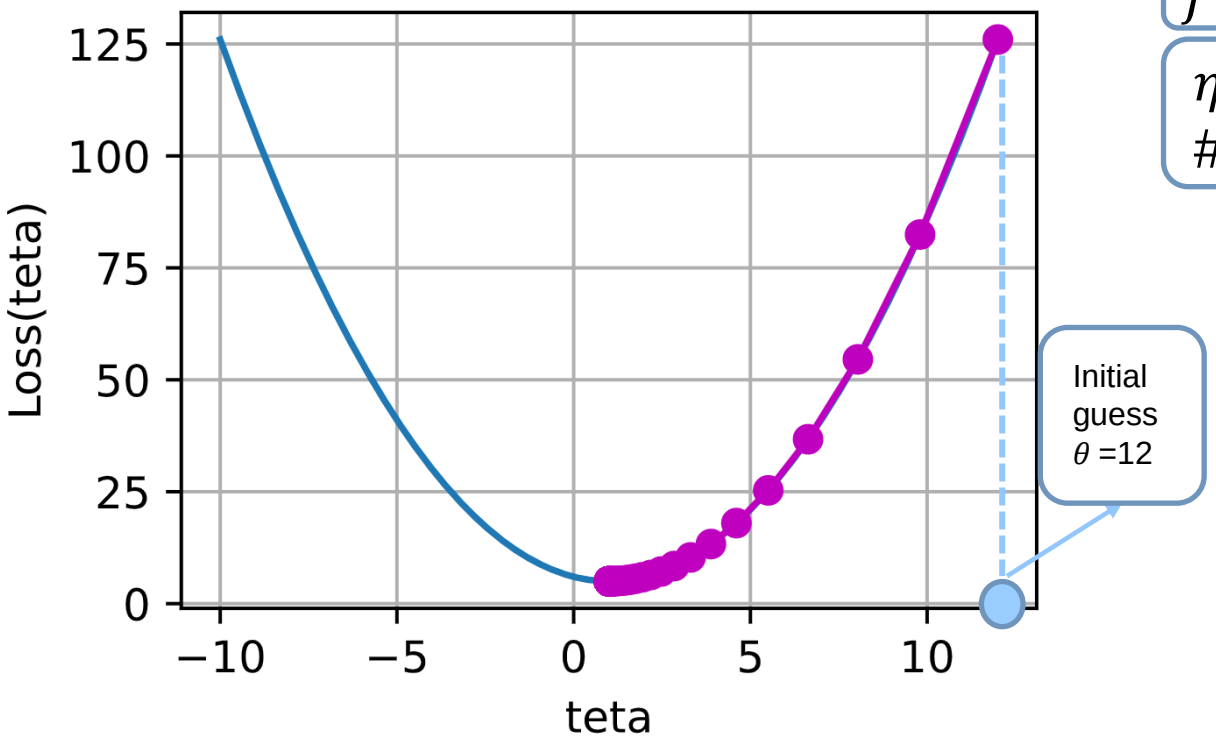
$$\theta_{new} = \theta_{old} - \eta f'(\theta = 12)$$
$$= 12 - 0.1 \times 22 = 9.8$$



$f(\theta) = (\theta - 1)^2 + 5$

$f'(\theta) = 2(\theta - 1)$

η =0.1
#of iterations = 200

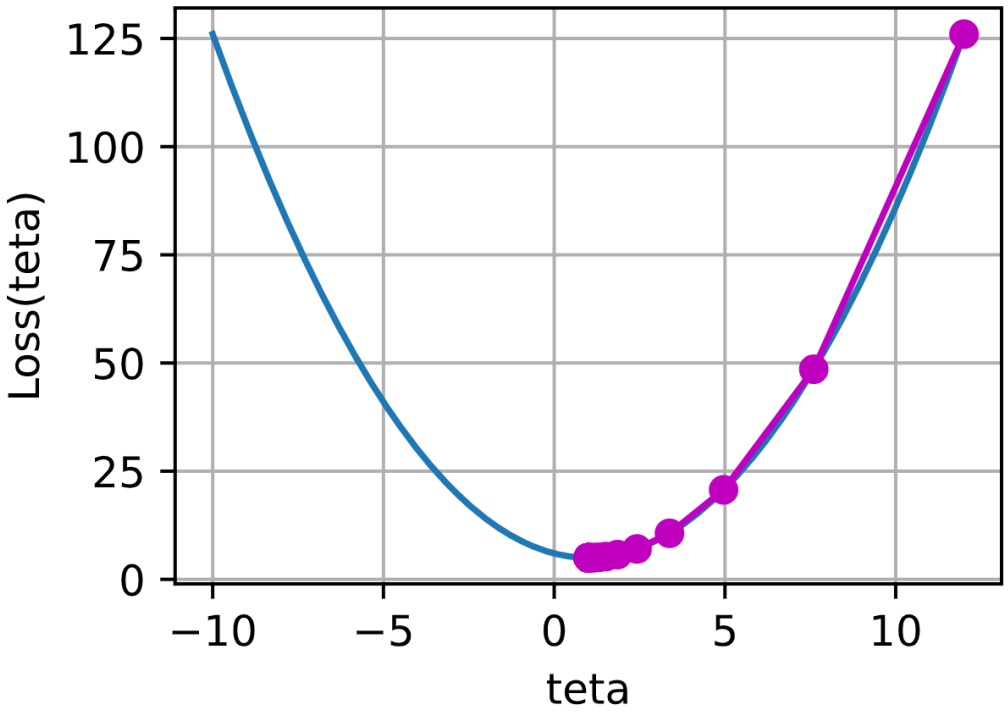


$f(\theta) = (\theta - 1)^2 + 5$

$f'(\theta) = 2(\theta - 1)$

η =0.2

#of iterations = 200

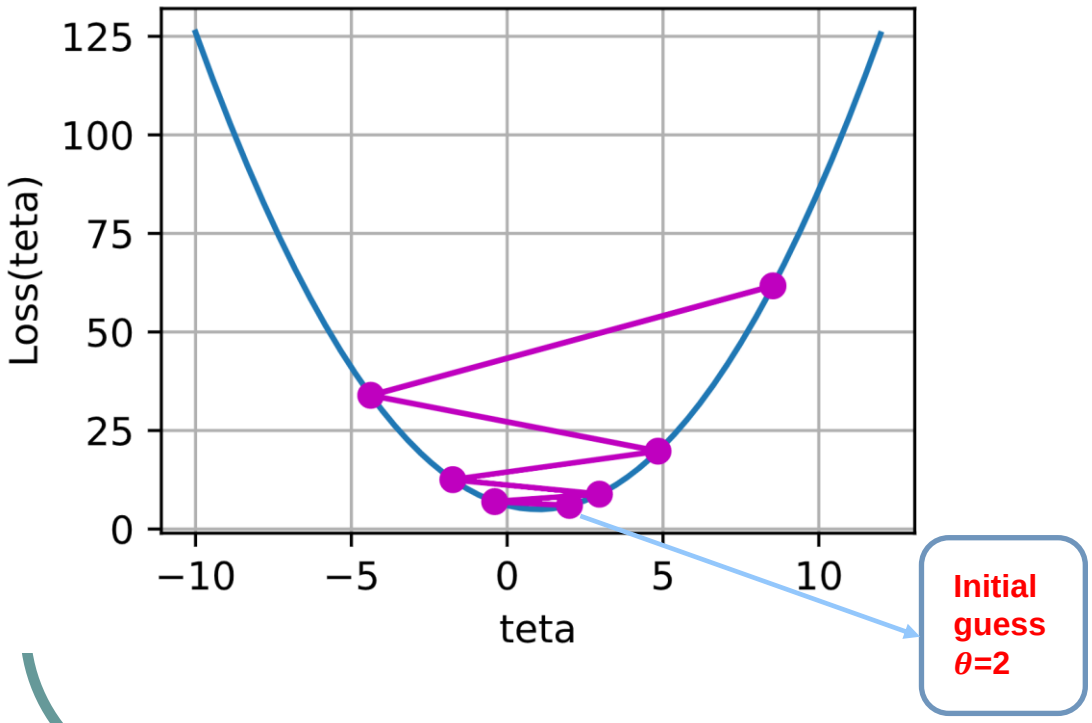


$f(\theta) = (\theta - 1)^2 + 5$

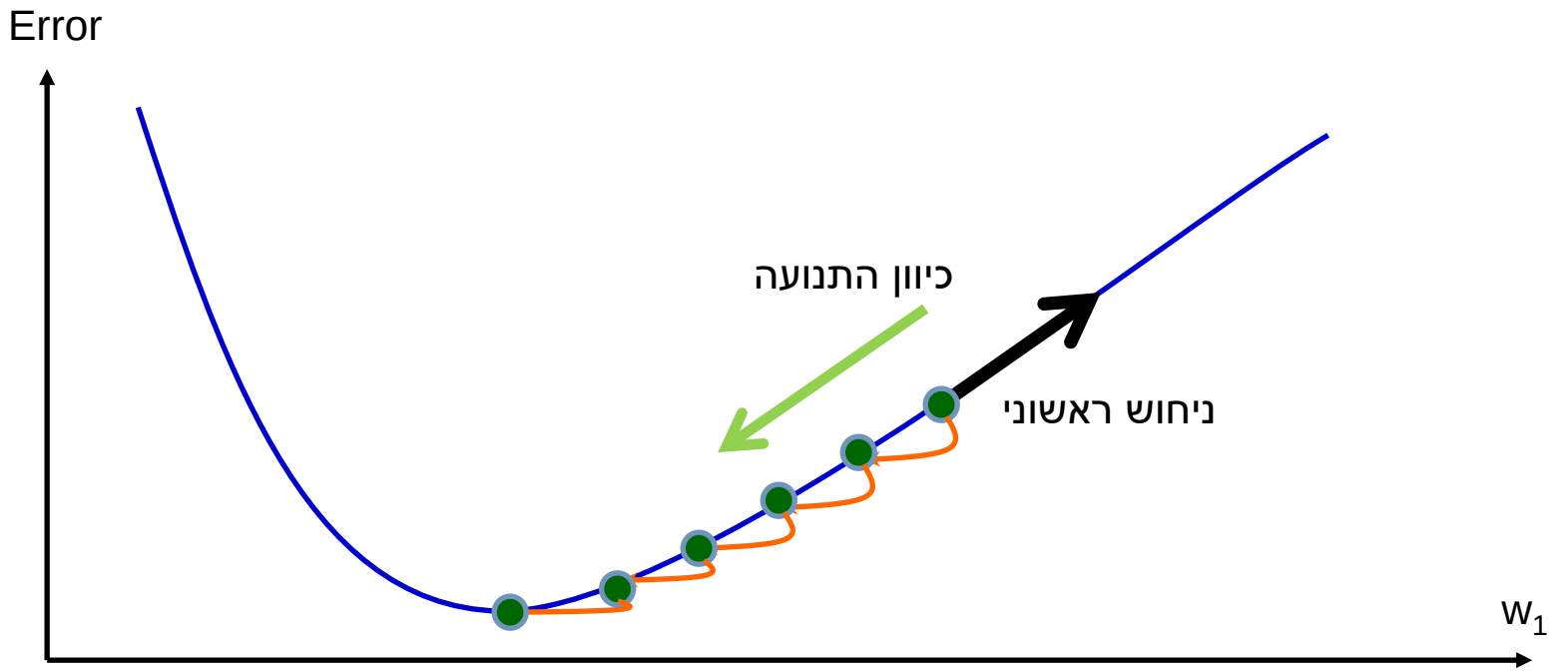
$f'(\theta) = 2(\theta - 1)$

η =**1.2**

#of iterations = 5



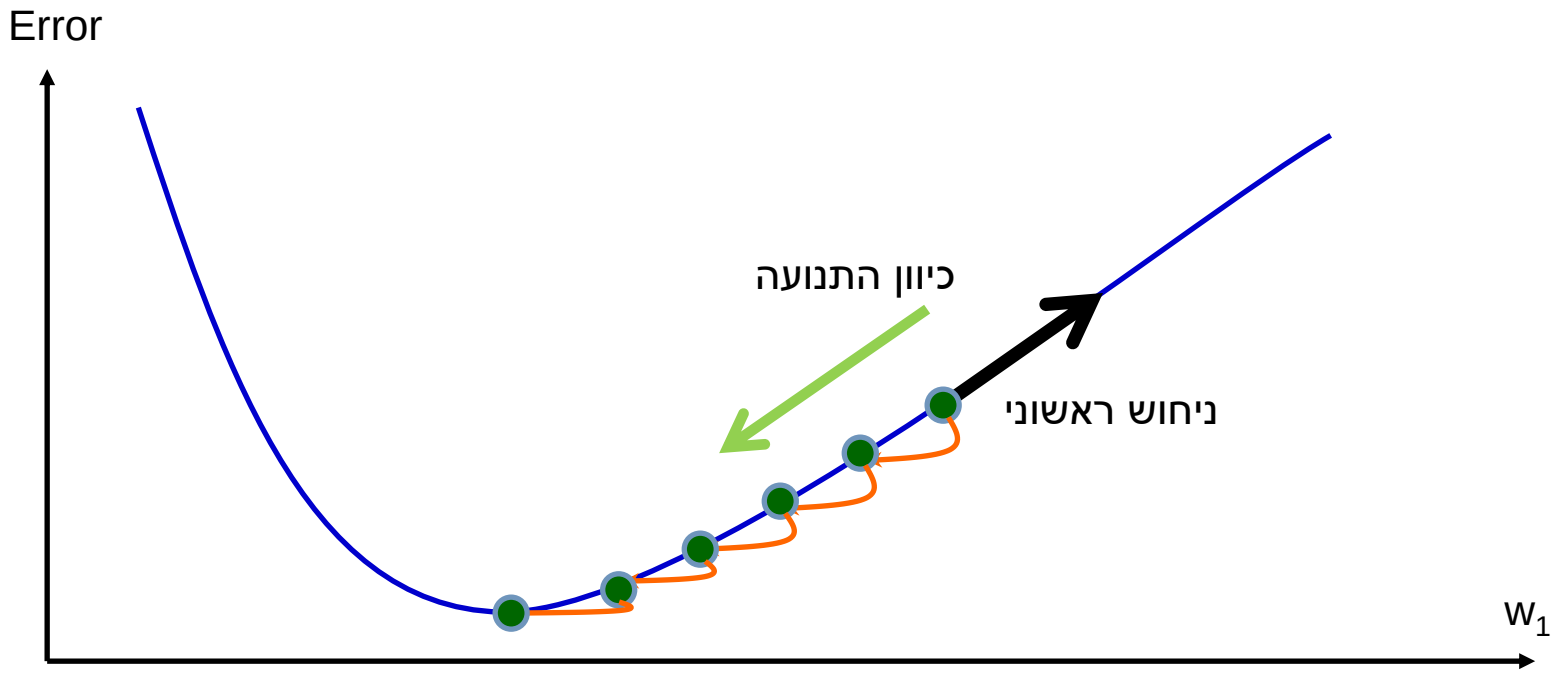
משתנה אחד – Gradient descent



הרעיון:

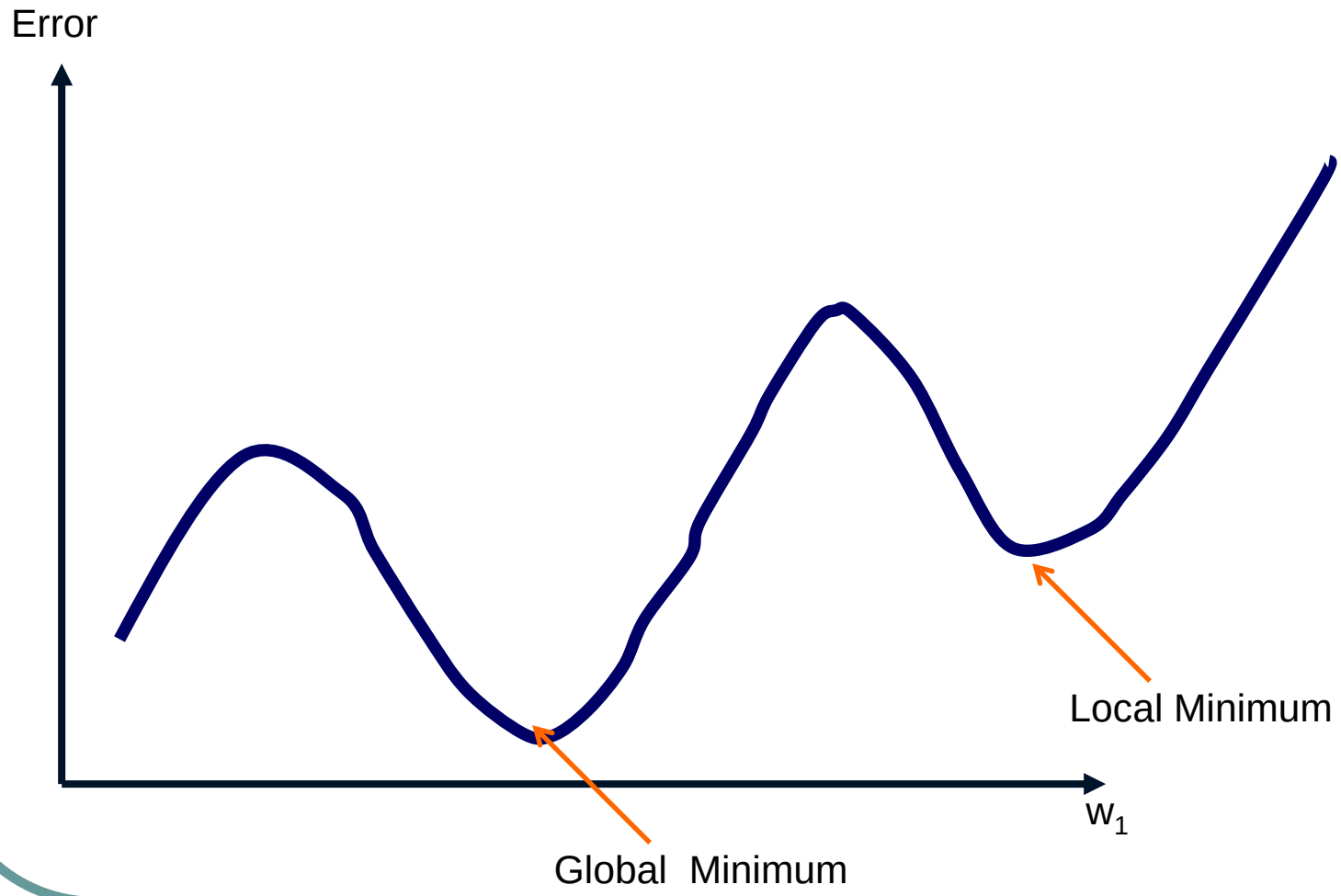
1. מצא את כיוון הגרדיאנט
2. חשב כיוון תנועה עפ"י 1
3. הגדר "צעד"
4. נוע בכיוון הצעד
5. חזור על סעיפים 1-4 עד שמתקבל ערך טעות הקטן מספיק

משתנה אחד – Gradient descent



צעד קטן מדי – לא יעיל מבחינה חישובית
צעד גדול מדי – עלול להביא לתוצאה לא נכונה

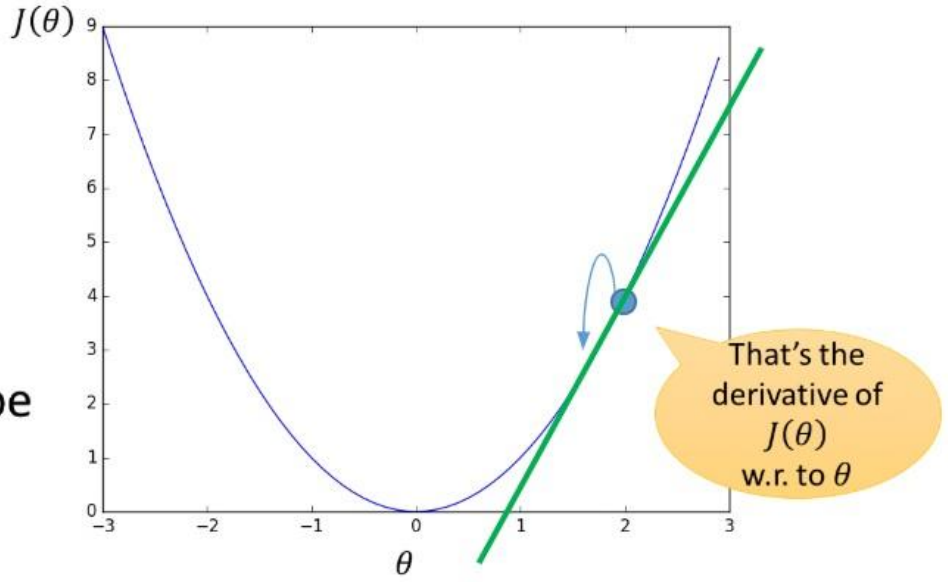
Local minima vs. Global minimum



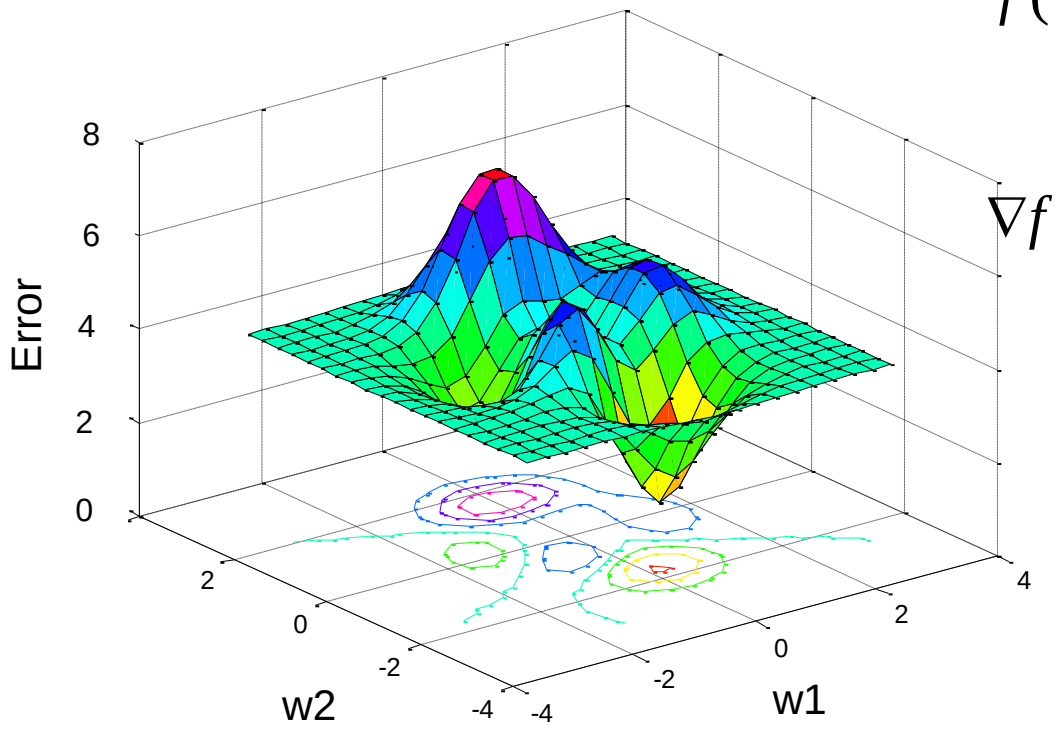
סיכום עד כה

Training as a minimization process

- Given a training set (dataset for finding the optimal model parameters), we usually minimize the **train-loss**, that is, the loss over the training set
- We use **Gradient Descent**
- We start with random values for θ, and then update them to minimize $J(\theta)$
- Update is done using the **opposite direction** of the slope of $J(\theta)$ at θ



בשני מימדים



$$f(w):\Re^2 \to \Re$$

$$\nabla f(w_1, w_2) = \begin{pmatrix} \frac{\partial}{\partial w_1} Error(w) \\ \frac{\partial}{\partial w_2} Error(w) \end{pmatrix}$$

נגזרות חלקיות וגרדיאנט

- **נגזרת חלקית** של פונקציה בכמה משתנים $\frac{\partial f}{\partial x_k}$ היא נגזרת של הפונקציה באחד ממשתניה, כאשר מתייחסים לשאר המשתנים כאל קבועים

- גרדיאנט – ווקטור הנגזרות החלקיות

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \ldots\ldots, \frac{\partial f}{\partial x_k} \right)$$

Gradient descent in *m* dimensions

Given $\mathrm{f}(\mathbf{w}) : \Re^m \rightarrow \Re$

$$\nabla \mathrm{f}(\mathrm{w}) = \begin{pmatrix} \frac{\partial}{\partial w_1} \mathrm{f}(\mathrm{w}) \\ \vdots \\ \frac{\partial}{\partial w_m} \mathrm{f}(\mathrm{w}) \end{pmatrix}$$ points in direction of steepest ascent.

GRADIENT DESCENT RULE: $\mathrm{w} \leftarrow \mathrm{w} - \eta \nabla \mathrm{f}(\mathrm{w})$

Equivalently $w_j \leftarrow w_j - \eta \frac{\partial}{\partial w_j} \mathrm{f}(\mathrm{w})$ ....where w_j is the jth weight

"just like a linear feedback system"

http://www.autonlab.org/tutorials/

$E = f(w)$

m – מס' המאפיינים המייצגים דוגמא $(x_1, x_2, \ldots\ldots, x_m)$

R – מס' הדוגמאות בקבוצת האימון

$\sum_{i=0}^{m} w_i x_i$

$$E = \sum_{i=1}^{R} (Prediction_{y_i} - y_i \ )^2$$

$$E = \sum_{i=1}^{R} (\sum_{i=0}^{m} w_i x_i - y_i \ )^2)$$

$$\frac{\partial E}{\partial w_i} = ?$$

$E = f(w)$

m – מס' המאפיינים המייצגים דוגמא $(x_1, x_2, \ldots\ldots, x_m)$

R – מס' הדוגמאות בקבוצת האימון

$\sum_{i=0}^{m} w_i x_i$

$$E = \sum_{i=1}^{R} (Prediction_{y_i} - y_i \ )^2$$

$$E = \sum_{i=1}^{R} (\sum_{i=0}^{m} w_i x_i - y_i \ )^2)$$

$$\frac{\partial E}{\partial w_i} = ?$$

$E = f(w)$

m – מס' המאפיינים המייצגים דוגמא $(x_1, x_2, \ldots\ldots, x_m)$

R – מס' הדוגמאות בקבוצת האימון

$$E = \sum_{i=1}^{R} (\ Prediction_{y_i} - y_i \ )^2$$

$E = f(w)$

m – מס' המאפיינים המייצגים דוגמא $(x_1, x_2, \ldots\ldots, x_m)$

R – מס' הדוגמאות בקבוצת האימון

$$\sum_{i=0}^{m} w_i x_i$$

$$E = \sum_{i=1}^{R} (Prediction_{y_i} - y_i \ )^2$$

$E = f(w)$

m – מס' המאפיינים המייצגים דוגמא $(x_1, x_2, \ldots\ldots, x_m)$

R – מס' הדוגמאות בקבוצת האימון

$\sum_{i=0}^{m} w_i x_i$

$$E = \sum_{i=1}^{R} (Prediction_{y_i} - y_i \ )^2$$

$$E = \sum_{i=1}^{R} (\sum_{i=0}^{m} w_i x_i - y_i \ )^2)$$

$$\frac{\partial E}{\partial w_i} = ?$$

Linear Perceptron training rule

$$E = \sum_{k=1}^{R} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

Gradient descent tells us we should update **w** thusly if we wish to minimize *E:*

$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

So what's $\frac{\partial E}{\partial w_j}$?

http://www.autonlab.org/tutorials/

Linear Perceptron training rule

$$E = \sum_{k=1}^{R} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

Gradient descent tells us we should update **w** thusly if we wish to minimize *E:*

$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

So what's $\frac{\partial E}{\partial w_j}$?

$$\frac{\partial E}{\partial w_j} = \sum_{k=1}^{R} \frac{\partial}{\partial w_j} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

http://www.autonlab.org/tutorials/

Linear Perceptron training rule

$$E = \sum_{k=1}^{R} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

Gradient descent tells us we should update **w** thusly if we wish to minimize *E:*

$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

So what's $\frac{\partial E}{\partial w_j}$?

$$\frac{\partial E}{\partial w_j} = \sum_{k=1}^{R} \frac{\partial}{\partial w_j} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

$$= \sum_{k=1}^{R} 2(y_k - \mathbf{w}^T \mathbf{x}_k) \frac{\partial}{\partial w_j} (y_k - \mathbf{w}^T \mathbf{x}_k)$$

http://www.autonlab.org/tutorials/

Linear Perceptron training rule

$$E = \sum_{k=1}^{R} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

Gradient descent tells us we should update **w** thusly if we wish to minimize *E:*

$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

So what's $\frac{\partial E}{\partial w_j}$?

$$\frac{\partial E}{\partial w_j} = \sum_{k=1}^{R} \frac{\partial}{\partial w_j} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

$$= \sum_{k=1}^{R} 2(y_k - \mathbf{w}^T \mathbf{x}_k) \frac{\partial}{\partial w_j} (y_k - \mathbf{w}^T \mathbf{x}_k)$$

$$= -2 \sum_{k=1}^{R} \delta_k \frac{\partial}{\partial w_j} \mathbf{w}^T \mathbf{x}_k$$

...where...

$$\delta_k = y_k - \mathbf{w}^T \mathbf{x}_k$$

http://www.autonlab.org/tutorials/

Linear Perceptron training rule

$$E = \sum_{k=1}^{R} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

Gradient descent tells us we should update **w** thusly if we wish to minimize *E:*

$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

So what's $\frac{\partial E}{\partial w_j}$?

$$\frac{\partial E}{\partial w_j} = \sum_{k=1}^{R} \frac{\partial}{\partial w_j} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

$$= \sum_{k=1}^{R} 2(y_k - \mathbf{w}^T \mathbf{x}_k) \frac{\partial}{\partial w_j} (y_k - \mathbf{w}^T \mathbf{x}_k)$$

$$= -2 \sum_{k=1}^{R} \delta_k \frac{\partial}{\partial w_j} \mathbf{w}^T \mathbf{x}_k$$

...where...

$$\delta_k = y_k - \mathbf{w}^T \mathbf{x}_k$$

$$= -2 \sum_{k=1}^{R} \delta_k \frac{\partial}{\partial w_j} \sum_{i=1}^{m} w_i x_{ki}$$

$$= -2 \sum_{k=1}^{R} \delta_k x_{kj}$$

http://www.autonlab.org/tutorials/

Linear Perceptron training rule

$$E = \sum_{k=1}^{R} (y_k - \mathbf{w}^T \mathbf{x}_k)^2$$

R – מס' הדוגמאות בקבוצת האימון

Gradient descent tells us we should update **w** thusly if we wish to minimize *E:*

$$w_j \leftarrow w_j - \eta \frac{\partial E}{\partial w_j}$$

...where...

$$\frac{\partial E}{\partial w_j} = -2 \sum_{k=1}^{R} \delta_k x_{kj}$$

$$w_j \leftarrow w_j + 2\eta \sum_{k=1}^{R} \delta_k x_{kj}$$

We frequently neglect the 2 (meaning we halve the learning rate)

http://www.autonlab.org/tutorials/

The “batch” perceptron algorithm

1) Randomly initialize weights w_1 w_2 … w_m

2) Get your dataset (append 1’s to the inputs if you don’t want to go through the origin).

3) for $i = 1$ to R $\quad \delta_i := y_i - \mathbf{w}^T \mathbf{x}_i$

4) for $j = 1$ to m $\quad w_j \leftarrow w_j + \eta \sum_{i=1}^{R} \delta_i x_{ij}$

5) if $\sum \delta_i^2$ stops improving then stop. Else loop back to 3.

http://www.autonlab.org/tutorials/

דוגמא

AND		
X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1



AND			
X0	X1	X2	Y
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

דוגמא

$$w_j \leftarrow w_j + 2\eta \sum_{k=1}^{R} \delta_k x_{kj}$$

$$\delta_k = y_k - \mathbf{w}^T \mathbf{x}_k$$

AND			
X0	X1	X2	Y
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

Iteration 0: w0 = 0.3, w1=0.4, w2=0.5 – ניחוש ראשוני

AND				out	Error
X0	X1	X2	Y		
1	0	0	0	0.3	δ1 = -0.3
1	0	1	0	0.8	δ2 = -0.8
1	1	0	0	0.7	δ3 = -0.7
1	1	1	1	1.2	δ4 = -0.2

מעבר על כל הדוגמאות וחישוב הטעות

$$E = (-0.3)^2 + (-0.8)^2 + (-0.7)^2 + (-0.2)^2 = 0.09$$

דוגמא

$$w_j \leftarrow w_j + 2\eta \sum_{k=1}^{R} \delta_k x_{kj}$$

$$\delta_k = y_k - \mathbf{w}^T \mathbf{x}_k$$

Iteration 0: w0 = 0.3, w1=0.4, w2=0.5

AND				out		Error
X0	X1	X2	Y			
1	0	0	0	0.3		δ1 = -0.3
1	0	1	0	0.8		δ2 = -0.8
1	1	0	0	0.7		δ3 = -0.7
1	1	1	1	1.2		δ4 = -0.2

$$\eta = 0.05$$

$$2\eta \sum_{k=1}^{4} \delta_k x_{k0} = 0.1 \times (-0.3 \times 1 + (-0.8) \times 1 + (-0.7) \times 1 + (-0.2 \times 1) = -0.2$$

$$2\eta \sum_{k=1}^{4} \delta_k x_{k1} = 0.1 \times (-0.3 \times 0 + (-0.8) \times 0 + (-0.7) \times 1 + (-0.2 \times 1) = -0.09$$

$$2\eta \sum_{k=1}^{4} \delta_k x_{k2} = 0.1 \times (-0.3 \times 0 + -0.8 \times 1 + -0.7 \times 0 + (-0.2 \times 1) = -0.1$$

דוגמא-AND

$$w_j \leftarrow w_j + 2\eta \sum_{k=1}^{R} \delta_k x_{kj}$$

$$\delta_k = y_k - \mathbf{w}^T \mathbf{x}_k$$

Iteration 0: w0 = 0.3, w1=0.4, w2=0.5

AND				out		Error
X0	X1	X2	Y			
1	0	0	0	0.3		
1	0	1	0	0.8		
1	1	0	0	0.7		
1	1	1	1	1.2		

$$2\eta \sum_{k=1}^{4} \delta_k x_{k0} = -0.2$$

$$2\eta \sum_{k=1}^{4} \delta_k x_{k1} = -0.09$$

$$2\eta \sum_{k=1}^{4} \delta_k x_{k2} = -0.1$$

$$w_0 = w_0 + \Delta w_0 = 0.3 + (-0.2) = 0.1$$

$$w_1 = w_1 + \Delta w_1 = 0.4 + (-0.09) = 0.31$$

$$w_2 = w_2 + \Delta w_2 = 0.5 + (-0.1) = 0.4$$

דוגמא-AND

$$w_j \leftarrow w_j + 2\eta \sum_{k=1}^{R} \delta_k x_{kj}$$

$$\delta_k = y_k - \mathbf{w}^T \mathbf{x}_k$$

Iteration 1: w0 = 0.1, w1=0.31, w2=0.4

AND				out	Error
X0	X1	X2	Y		
1	0	0	0	0.1	$\delta 1 = -0.1$
1	0	1	0	0.5	$\delta 2 = -0.5$
1	1	0	0	0.41	$\delta 3 = -0.41$
1	1	1	1	0.81	$\delta 4 = -0.19$

$$E = (0.1)^2 + (0.5)^2 + (0.41)^2 + (0.19)^2 = 0.0361$$

$$0.0361 < 0.9$$

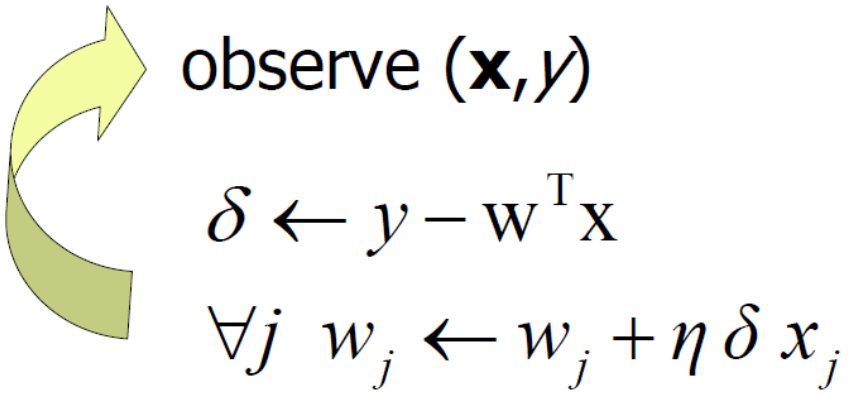
נמשיך לעוד איטרציה.....

We can use the algorithm in incremental mode

קיימות מס 'צורות הפעלה לאלגוריתם:

1) מוד Gradient Descent GD (מכונה לעיתים Batch)
2) מוד Stochastic Gradient Descent SGD (מכונה לעיתים Increnmetal)
3) מוד Mini-Batches

SGD Mode



http://www.autonlab.org/tutorials/

Gradient Descent (GD)

- Gradient descent - update weights only after accumulating gradients for all training-set instances

```
for e in range(epoch_number):
        for x,y in training set:
                Calculate Delta
                Update Weights
```

$O(md)$

$O(d)$

Stochastic Gradient Descent (SGD)

- **Stochastic** Gradient descent - update weights after every instance. Usually - shuffle the training set before each epoch

```
for e in range(epoch_number):
        shuffle(training_set)
        for x,y in training_set:
            Update Weights
```

O(*m*)

O(*md*)

פונקציית הטעות צריכה להיות מוגדרת עפ"י הסיווג

Don't minimize $\sum \left(y_i - \mathrm{w}^{\mathrm{T}} \mathrm{x}_i\right)^2$.

Minimize number of misclassifications instead

$$\sum \left(y_i - \text{Sign}\left(\mathrm{w}^{\mathrm{T}} \mathrm{x}_i\right)\right)^2$$

where Sign $(x) = \begin{cases} -1 \text{ if } x<0 \\ 1 \text{ if } x \geq 0 \end{cases}$

The gradient descent rule can be changed to:

if $(\boldsymbol{x}_j, y_j)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_j$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_j$

http://www.autonlab.org/tutorials/

דוגמא SGD

- דוגמא:
  - חישוב משקלי פרספטרון
  - מוד אינקרמנטלי
  - נתחיל ממשקלים אקראיים: w_0=-0.5, w_1 = 2, w_2=4

AND		
X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1



AND			
X0	X1	X2	Y
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	1

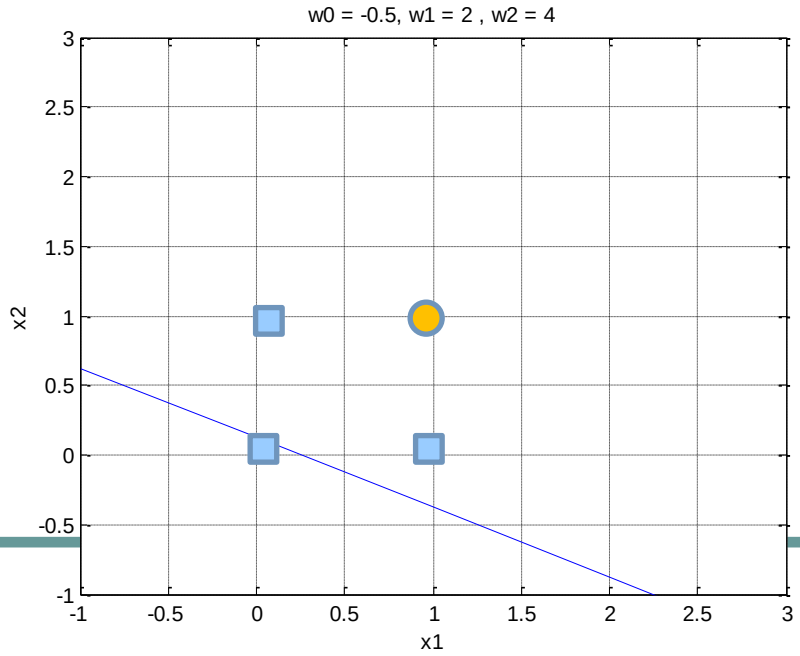
if (x_i, y_i) correctly classed, don't change

if wrongly predicted as 1 $\quad w \leftarrow w - x_i$

if wrongly predicted as -1 $\quad w \leftarrow w + x_i$

דוגמא SGD

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-0.5	2	4	-0.5	NEG	no



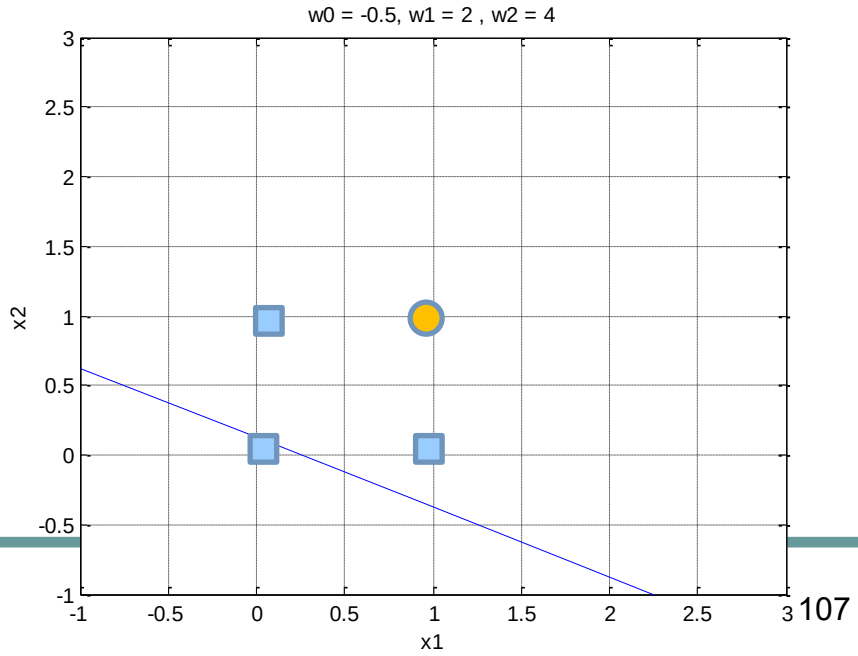
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-0.5	2	4	-0.5	NEG	no
1	0	1	0	-0.5	2	4	3.5	POS	yes



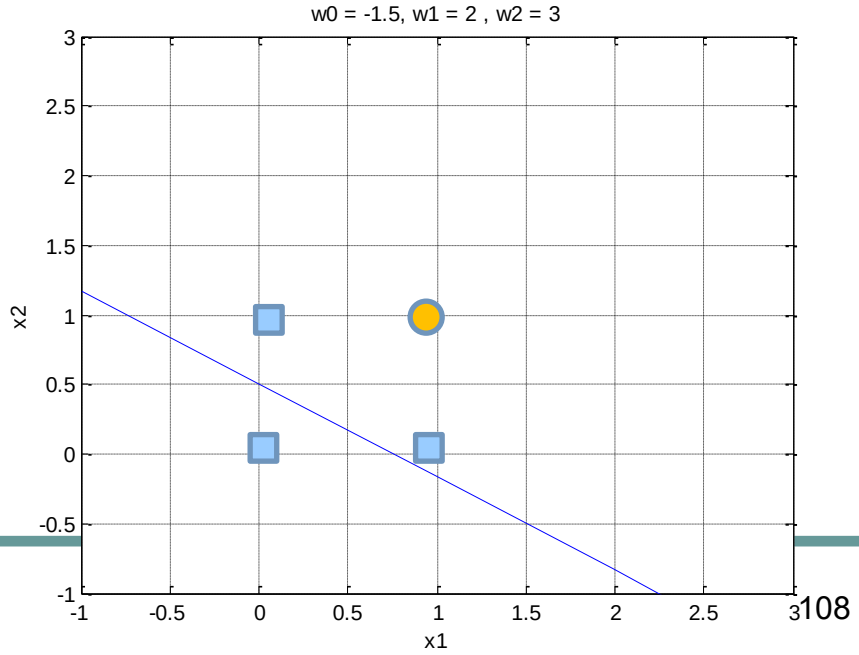
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-0.5	2	4	-0.5	NEG	no
1	0	1	0	-0.5	2	4	3.5	POS	yes
1	1	0	0	-1.5	2	3	0.5	POS	yes



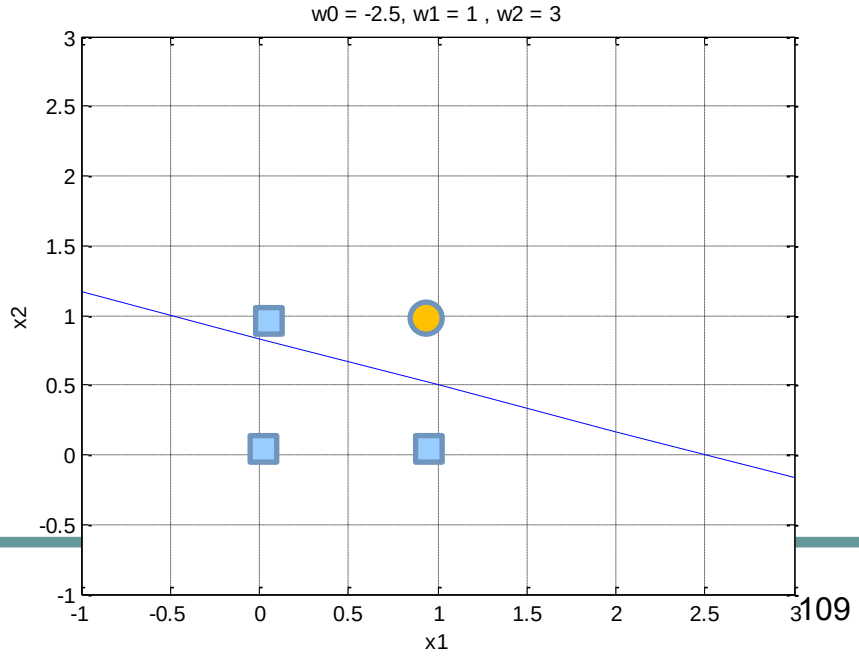
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-0.5	2	4	-0.5	NEG	no
1	0	1	0	-0.5	2	4	3.5	POS	yes
1	1	0	0	-1.5	2	3	0.5	POS	yes
1	1	1	1	-2.5	1	3	1	POS	no



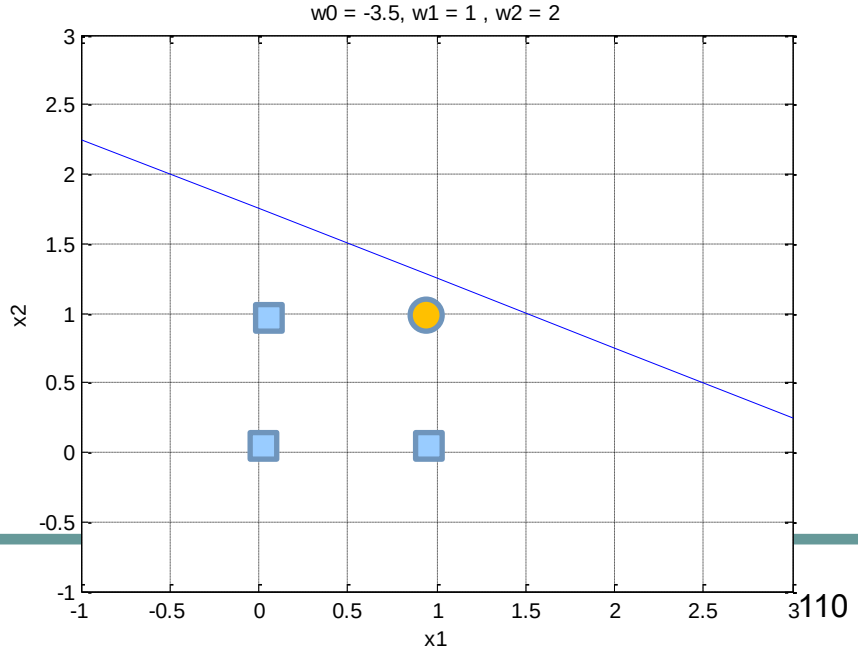
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-2.5	1	3	-2.5	NEG	no
1	0	1	0	-2.5	1	3	0.5	POS	yes
1	1	0	0	-3.5	1	2	-2.5	no	no



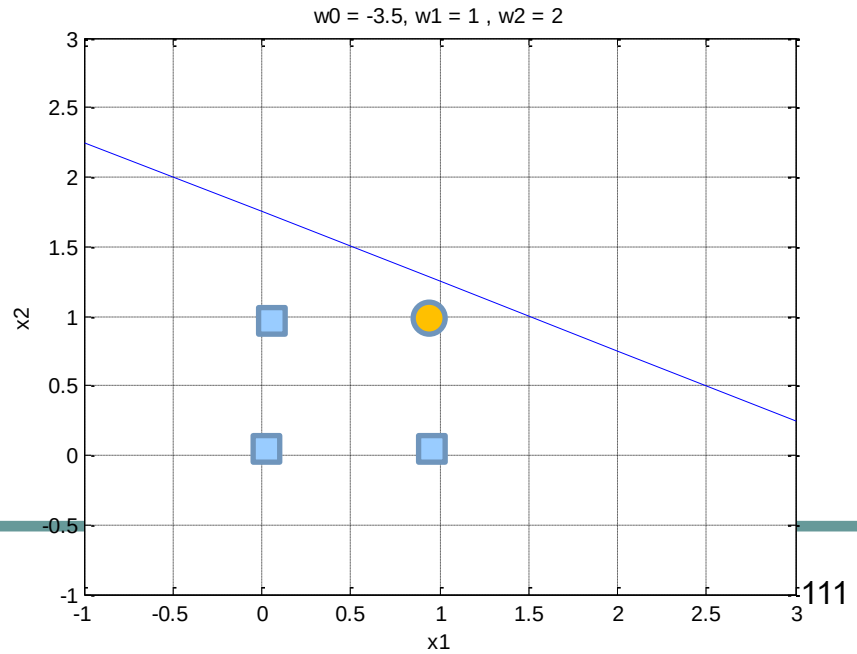
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-2.5	1	3	-2.5	NEG	no
1	0	1	0	-2.5	1	3	0.5	POS	Yes
1	1	0	0	-3.5	1	2	-2.5	NEG	No
1	1	1	1	-3.5	1	2	-0.5	NEG	yes



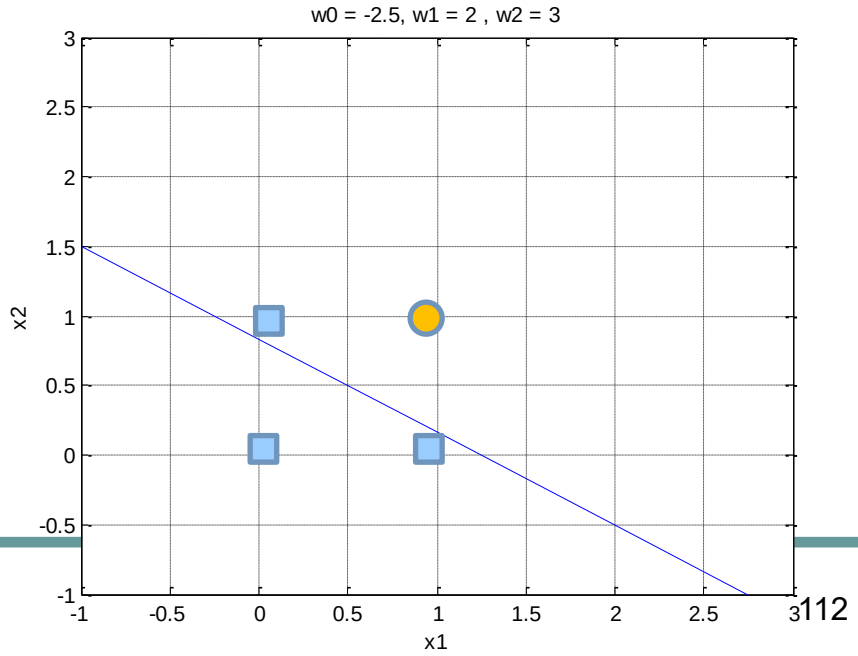
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-2.5	2	3	-3.5	NEG	no
1	0	1	0	-2.5	2	3	0.5	POS	yes
1	1	0	0						
1	1	1	1						



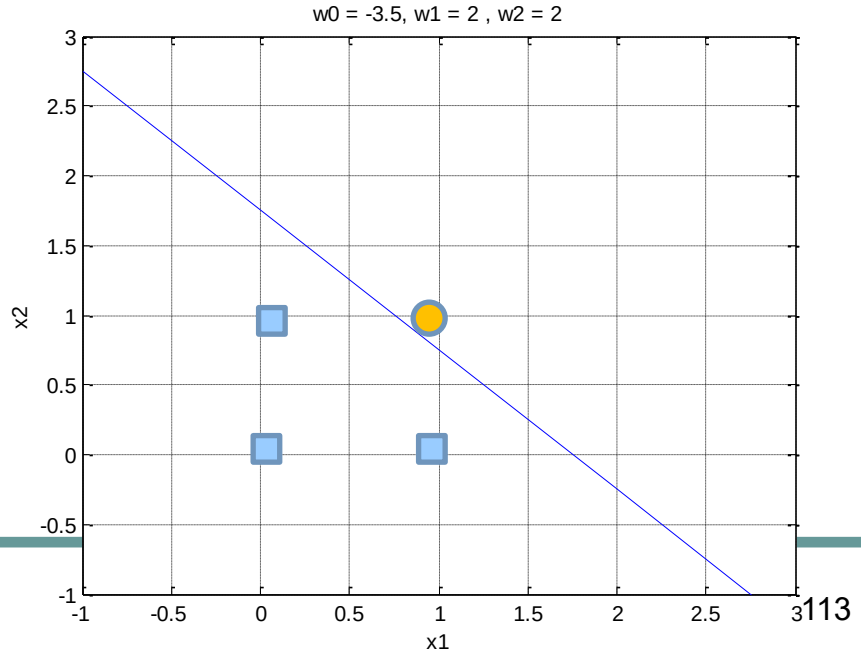
if $(\boldsymbol{x}_i, y_i)$ correctly classed, don't change

if wrongly predicted as 1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} - \boldsymbol{x}_i$

if wrongly predicted as -1 $\quad \boldsymbol{w} \leftarrow \boldsymbol{w} + \boldsymbol{x}_i$

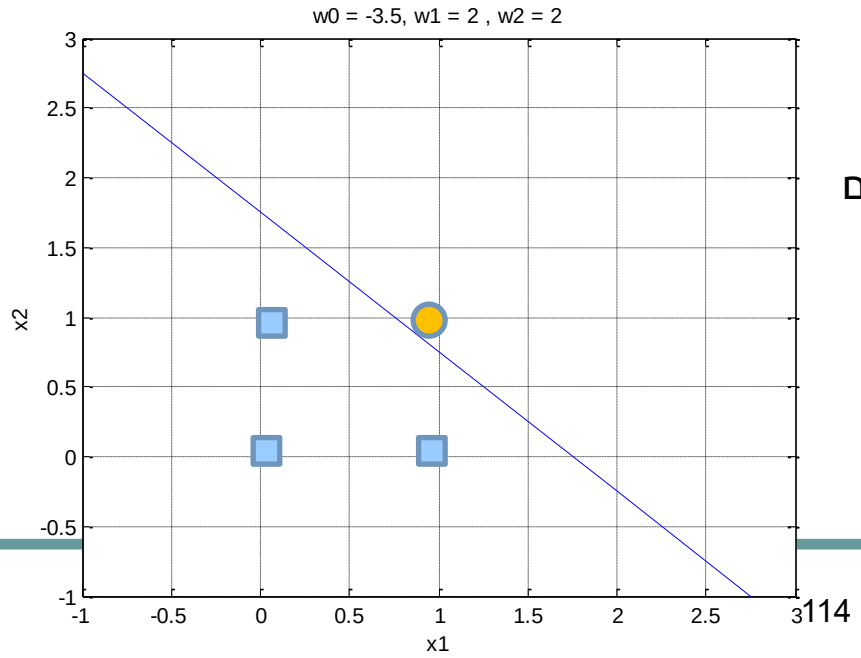
AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-2.5	2	3	-3.5	NEG	no
1	0	1	0	-2.5	2	3	0.5	POS	yes
1	1	0	0	-3.5	2	2	-1.5	NEG	no
1	1	1	1	-3.5	2	2	0.5	POS	no



AND

AND				W					
X0	X1	X2	Y	w0	w1	w2	wx	Prediction	Update?
1	0	0	0	-3.5	2	2	-3.5	NEG	no
1	0	1	0	-3.5	2	2	-1.5	NEG	no
1	1	0	0	-3.5	2	2	-1.5	NEG	no
1	1	1	1	-3.5	2	2	0.5	POS	no



נמשיך בתהליך עד
שנגיע ל"סיבוב" שלם
ללא צורך בעידכון

משפט ההתכנסות של פרספטרון

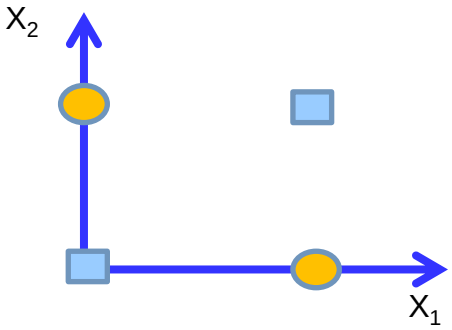
- הנח ששתי המחלקות N,P ניתנות להפרדה לינארית ע"י סט המשקולות w* (כלומר ישנו על-מישור hyperplane המסוגל להפריד ביניהם), אזי אלגוריתם הפרספטרון יתכנס לאחר מס' צעדים סופי לסט משקולות w' (לא בהכרח שקול ל-w*) המפריד בין שתי המחלקות.

Perceptron

- פרספטרון (בעל שכבת כניסה אחת ושכבת יציאה אחת, ללא שכבות חבויות) יכול לסווג נכון רק בעיות הניתנות להפרדה לינארית.
  - AND ניתן להפרדה לינארית
  - XOR לא ניתן להפרדה לינארית
- רשת נוירונים עם שכבות חבויות ופונקציית "הפעלה" יכולה ללמוד ולסווג בעיות מורכבות יותר (שאינן ניתנות להפרדה לינארית)

XOR problem

X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0



XOR problem

X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0

ייצוג שונה של המאפיינים

$AND(\overline{x1}, x2)$	$AND(x1, \overline{x2})$	Y
0	0	0
1	0	1
0	1	1
0	0	0

